大学信息技术

计算思维与算法

目 录

1. 计算思维
2. 问题与求解
3. 算法
4. 排序算法
5. 常用算法

1. 计算思维

什么是计算思维?

- **推动人类文明进步和科技发展的三大科学支柱**
  - 理论科学
  - 实验科学
  - 计算科学
- **人类认识世界和改造世界的三种思维**
  - 逻辑思维
    以推理和演绎为特征，以数学学科为代表。
  - 实证思维
    以观察和总结自然规律为特征，以物理等学科为代表。
  - 计算思维
    以设计和构造为特征，以计算机学科为代表。

什么是计算思维?

计算思维是运用计算机科学的基础概念去求解问题、设计系统和理解人类行为，它涵盖了计算机科学的一系列思维活动。

- 问题求解中的计算思维
- 系统设计中的计算思维
- 人类行为理解中的计算思维

计算思维如同所有人都具备“读、写、算”能力一样，成为适合于每个人的一种普遍的认识和一类普适的技能。

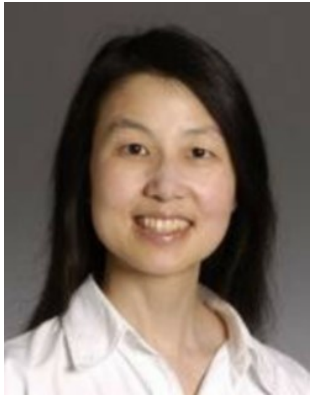


图: 周以真
卡内基梅隆大学教授

理解计算思维，首先要理解计算

理解计算思维的前提是理解计算，研究在解决问题过程中，哪些是可计算的，以及如何计算。

- 算术运算
- 集合运算
- 逻辑运算
- 条件运算
- ……

当我们跳出算术运算的局限，理解了计算的本质后，就会发现原来好多看似不可计算的东西都能变得可计算，也就很容易理解计算思维的普适性了。

计算思维的本质

- **抽象**
  把实际问题抽象为数学问题，并建模
  - 问题建模：忽略某些细节，如最短路径求解，方程计算。
  - 数据建模：考虑数据特点，数据可抽象为线性结构，树结构，图结构等。
- **自动化**
  机械地一步一步地自动执行，并选择合适的计算机执行问题的抽象。

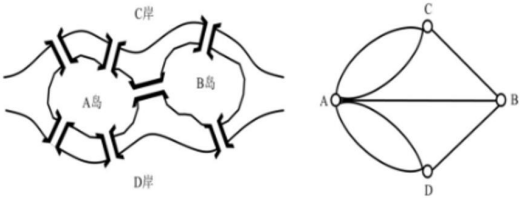


图: 哥尼斯堡的七座桥

例

大象放冰箱里分几步？

问题求解中的计算思维

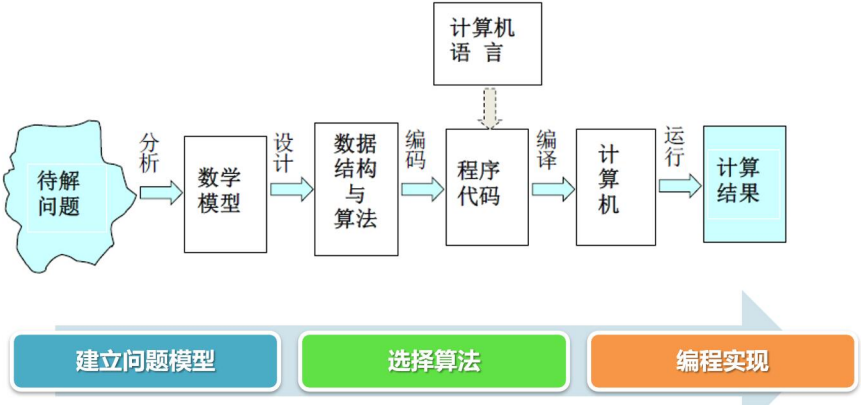


图: 使用计算机进行问题求解的计算思维过程

系统设计中的计算思维

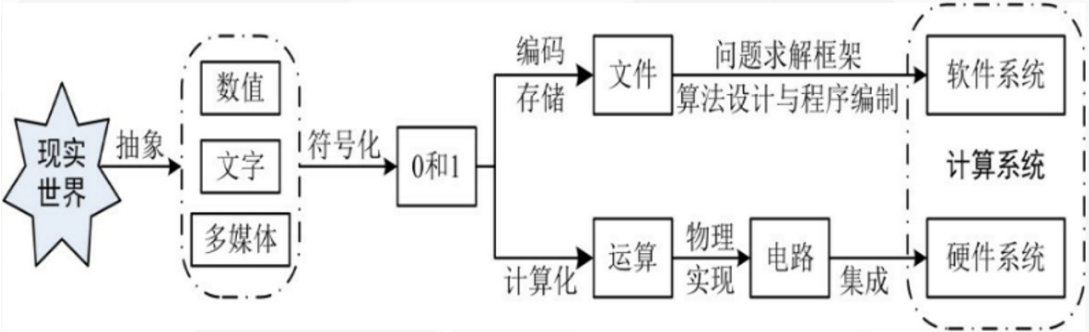


图: 将系统中的变换过程映射为符号变换—形式化，然后用计算机进行处理。

系统设计中的计算思维

我们经常在较低抽象级之上再建立一个较高的抽象级，以便隐藏低抽象级的复杂细节，提供更简单的求解方法。

如，对计算本身的理解就可以形成“电子电路 → 门逻辑 → 二进制 → 机器语言指令 → 高级语言程序”这样一个由低到高的抽象层次，我们之所以在高级语言程序这个层次上学习计算，当然是为了隐藏那些低抽象级的繁琐细节。

又如，在互联网上发送一封电子邮件实际上要经过不同抽象级的多层网络协议才得以实现，写邮件的人肯定不希望先掌握网络低层知识才能发送邮件。

人类行为理解中的计算思维

计算思维不仅能够反映人类思维活动，还能够通过计算手段来研究人类的行为。

利用计算手段来研究人类的行为，可视为社会计算，即通过各种信息技术手段，设计、实施和评估人与环境之间的交互。

- 网购习惯
- 推荐算法

计算思维理论的发展

计算思维 1.0	计算思维 2.0
计算思维是计算机科学家的思维模式	计算思维是所有领域的科学家在应用计算和计算模型时所采取的思维模式
计算思维是从计算机科学中产生，逐步扩散到其他科学领域	每个科学领域都有属于自身特点的计算思维，贯穿通过计算模型求解的设计、执行和评价过程
学习编程能力过程中产生计算思维，计算思维主要是大量的编程技巧	计算思维提供了有关模型和算法的丰富概念，学习和理解计算思维用以产生科学的编程能力
算法必须与程序紧密联系	算法并不一定要联系程序和计算机，算法是信息处理的技术，一些生物体内部进行的自然计算也属于算法
实施算法设计要求具备计算机科学的领域知识，特别是关于计算机的基本知识	每个人都可以在本领域找到问题计算求解的模型、算法和操作，并不需要图灵机这样的通用概念
应该关心程序背后的计算机概念而不是程序本身	应该关心程序背后反映领域特点的内容，注重本学科解决问题的计算化趋势

计算思维的特征

- 概念化，不是程序化
- 基础的，不是机械的技能
- 人的，不是计算机的思维
- 数学和工程思维的互补与融合
- 是思想，不是人造品
- 不能超出计算机的能力范围

不能超出计算机的能力范围

计算机的能力：能够高速执行大量指令

计算机的限制：每条指令只能进行有限的一些简单操作

例

某同学刚学了几天程序语言设计，想试试自己的能力，求方程 $3x^2 + 5x - 15 = 0$ 的根。该同学写出下述的程序段:

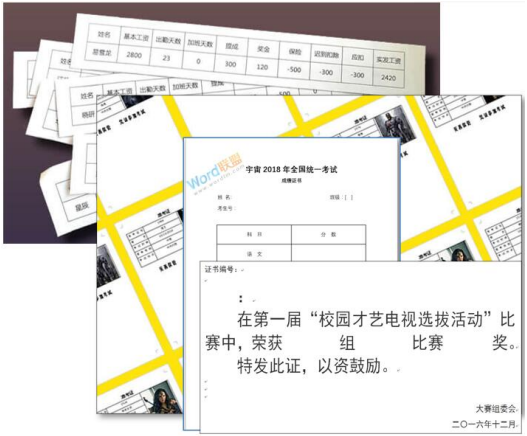
```
3*x*x + 5*x - 15 = 0; # 给出方程，让计算机求解
print(x); # 输出结果
```

```python
a = float(input("输入a:"))
b = float(input("输入b:"))
c = float(input("输入c:"))
#计算delta
delta = b**2-4*a*c
#判断解的个数
if delta < 0:
    print("方程无解！")
elif delta == 0:
    x = b/(-2*a)
    print("x1=x2=",x)
else:
    x1 = (b+delta**0.5)/(-2*a)
    x2 = (b-delta**0.5)/(-2*a)
#输出x1,x2
print("x1=",x1)
print("x2=",x2)
```

方程$ax^2+bx+c=0$的解为

$$\frac{-b\pm\sqrt{b^2-4ac}}{2a}$$

计算思维案例：word 邮件合并



工资条，准考证，成绩单，证书等一类问题

1. 抽象
   对共同的特征进行抽取，建立模型
   - 文档固定模版：统一风格
   - 文档内容分为固定内容和动态内容
   - 动态内容具有相同数据结构
2. 自动化
   - 做好数据源，文档模板
   - 将数据每个属性写入到文档模板
   - 重复步骤 B，合并所有数据

计算思维对其他学科的影响

- 数学
- 化学
- 生物学
- 社会学
- 博弈论

计算数学

计算机对数学来说过去只是一个数值计算工具，用于快速、大规模的数值计算，对数值计算方法的研究导致了计算数学的形成。

后来数学家利用计算机进行代数演算，形成了计算机代数；利用计算机研究几何问题，形成了计算几何学。

数学家还利用计算机去验证数学猜想，虽然不能证明猜想，但是一旦发现反例就可以推翻猜想，以免数学家毕生投入到一个不成立的猜想之中。

总之，现在计算机已经成为数学的研究手段，大大扩展了数学家的能力。

机器证明

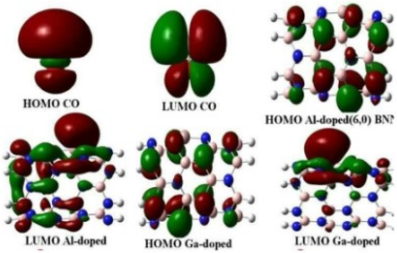
机器证明是使用计算机证明定理，也称为定理的机械证明或自动证明。

70 年代后期，在计算机技术大发展的背景下，吴文俊继承和发展了中国古代数学的传统（即算法化思想），转而研究几何定理的机器证明，彻底改变了这个领域的面貌，是国际自动推理界先驱性的工作，被称为“吴方法”，产生了巨大影响。

图: 中科院资深院士，中国人工智能之父—吴文俊

计算化学

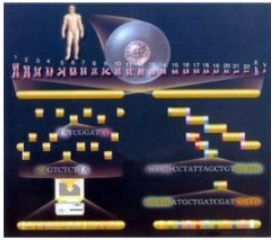
- 数值计算。对化学各专业的数学模型进行数值计算或方程求解
- 化学模拟，数值模拟、过程模拟和实验模拟
- 在有机分析中根据图谱数据库进行图谱检索等
- 利用原子计算去探索化学现象
- 用优化和搜索算法寻找优化化学反应条件和提高产量的物质

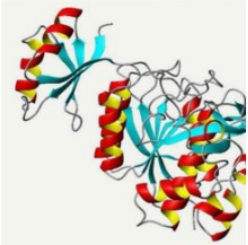


Gaussian 电荷分布的 HOMO 和 LUMO 轨道图

计算生物学

- 霰弹枪算法大大降低了人类基因组测序的成本，提高了测序的速度；
- 利用绳结来模拟蛋白质结构，用计算过程来模拟蛋白质动力学，并且运用数据挖掘与聚类分析的方法进行蛋白质结构的预测；
- 开发了生物数据处理分析方法和知识库，帮助人们从分子层次上认识生命的本质及其进化规律。
- 在医学领域，机器人手术、借助于计算机的分析诊断及可视化系统在临床中已经广泛应用。





谷歌 AlphaFold 解决蛋白质折叠问题

蛋白质的基本组成单位是氨基酸。蛋白质会由所含氨基酸残基的亲水性、疏水性、带正电、带负电等特性通过残基间的相互作用而折叠成一个立体的三级结构。

这个折叠后的三维形状至关重要，因为它决定了蛋白质的工作方式。如果开发新药物的科学家知道蛋白质的形状，他就更容易找到一个可以与之结合的分子，使这个蛋白质可以改变它的行为。问题在于，很难预测蛋白质将会呈现哪种形状。

1972 年的诺贝尔化学奖得主安芬森提出的著名假设，蛋白质分子的一级结构决定其立体结构。也就是说，如果从蛋白质的一级结构就能知道它的三级立体结构，那么就可以直接从基因推测其编码蛋白质所对应的生物学功能。

AlphaFold 系统已经能够以惊人的速度和准确性预测出氨基酸串会折叠成什么样子的立体形状。

计算博弈论

博弈论是一种数学工具，它允许我们对特定环境下的利己主义和理性行为者的战略互动进行推理。该结构提供了一组框架，描述了在这样一个战略代理人的环境下的理性结果。虽然博弈论领域起源于经济文献，但计算机科学家在过去几十年里从建模和计算的角度对这一领域做出了重大贡献，这导致了计算博弈论 (Algorithmic game theory)。

和传统的博弈论和计算机科学相比，算法博弈论主要关注点在互联网网络，非传统拍卖和人类行为的根本问题

- 社交网络里的个体行为
- 用拍卖的方式出售关键字广告位

计算社会学

计算社会学是当代社会学界借助计算机、互联网与人工智能技术等现代科技手段，利用大数据、新方法来获取数据与分析数据，从而研究与解释社会的一种新的范式或思维方式。计算社会学使用密集演算的方法来分析与模拟社会现象。

根据社会科学理论，以计算技术为工具，可以研究人类社会的组成、关系、结构、层次、行为、运动等问题；研究社会媒体，促进社会人群的交流与合作。

- 如何通过数据预测美国总统大选的结果？
- 流行病的发展轨迹

2. 问题与求解

Section 2.1

问题求解方法

问题及形式化描述

在各个学科领域，从自然学科到社会科学，从科学研究到生产实践，都存在着各种各样的问题。人们的活动都是一个不断提出问题、发现问题和解决问题的过程。

> 所谓问题，就是不能用已有的知识经验直接加以处理并因此而感到疑难的情境。例如，用六根火柴搭出四个等边三角形，三角形的边长等于一根火柴的长度，如何搭放？

问题一般具有 3 个基本成分：

(1) 一组已知的关于问题条件的描述，也即问题的起始状态；

(2) 关于构成问题结论的描述，也即问题要求的答案或目标状态；

(3) 问题解决过程中所遇到的困难。

问题抽象与数学模型

抽象（abstraction）是用于问题表示的重要思维工具。一般意义上的抽象，就是指忽略研究对象的具体的或无关的特性，而抽取其一般的或相关的特性。

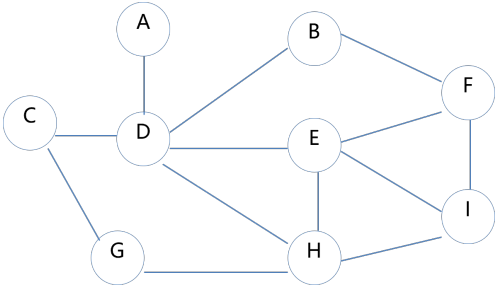
例

小学生经过学习都知道将应用题“原来有五个苹果，吃掉两个后还剩几个”抽象表示成“5 - 2”，这里显然只抽取了问题中的数量特性，完全忽略了苹果的颜色或吃法等不相关特性。

计算机科学中的抽象包括数据抽象和控制抽象，简言之就是将现实世界中的各种数量关系、空间关系、逻辑关系和处理过程等表示成计算机世界中的数据结构（数值、字符串、列表、堆栈、树等）和控制结构（基本指令、顺序执行、分支、循环、模块等），或者说建立实际问题的计算模型。

例

在土地确权时，需要标记每一农户的地界，并且需要标记地块的相邻关系。对于这个问题，即要考虑区域边界，还要考虑区域之间的相邻关系，经过抽象后，用带有字母的圆圈表示区域，并且在这个节点存储地块的地理位置，用连线表示区域之间的相邻关系，最后得到如图所示的图形。



数学模型及建模步骤

数学模型就是对实际问题的数学抽象后用数学符号、数学式子、程序、框图等对实际问题本质属性的刻画，用以描述客观事物的特征、内在联系及发展和运动规律。

1. 模型准备
2. 模型假设
3. 模型建立
4. 模型求解
5. 模型分析
6. 模型检验
7. 模型应用

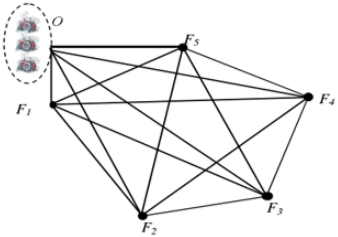
计算思维农业案例—农机调配问题

例

目前我国农机实行跨区作业服务模式，农户希望农田能及时颗粒归仓，机手希望在有限的时间内为尽可能多的农户服务，从而得到更多的收益。然而机手需要综合农田位置、面积、距离、天气、路况等多种因素，以作业收益最大为目标，来求解最佳的农机调配规划方案，以实现农机作业收益最大化问题。

问题抽象：

如图所示，节点 O 为机主的农机资源所在的位置，节点 F_1，F_2，F_3，F_4，F_5 表示机主所选的各个农田作业点的位置。每个农田作业点的面积、作业价格、所在位置、适宜服务的时间范围以及图中任意两点间的距离已知。

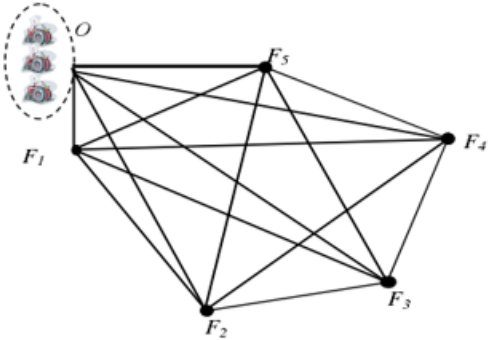


计算思维农业案例—农机调配问题

问题建模：

$$\mathrm{MaxP} = \sum_{j=1}^{n} p_j A_j - \sum_{j=0}^{n} T_{i,j} - \sum_{j=1}^{n} \mathrm{w}_j$$

其中 p_j 表示对于农田块 j 单位面积可得到的收入，$j = 1, 2, \cdots n$，A_j 表示农田块 j 的待作业面积；$T_{i,j}$ 表示由地点 i 到 j 的运输成本；w_j 表示农田块 j 的作业成本。



Section 2.2

计算机问题求解方法-算法

人类处理一般问题的特点

同学 A 和同学 B 约好周末从学校出发，去沧州动物园游玩。A 主张从学校门口叫出租车，B 则建议通过叫车软件网络约车。

1. 可选择的步骤非常多
2. 列出的步骤一般并不严格
3. 有些步骤还可以前后交换次序，甚至同时进行。

控制机器人

用计算机语言编制一个机器人行走的程序。

其中机器人可以执行的指令只有如下几个：

- 起立
- 坐下
- 向前走一步（必须在站立时执行）
- 向右转 90 度（必须在站立时执行）
- 举起手臂（向前抬到与身体成直角）
- 放下手臂

机器人面向墙坐在椅子上，椅子距离墙三步长。要求编写出一个具体的程序（步骤），让机器人向前一直走到墙，然后再走回来坐到椅子上。

指令序列

1. 起立
2. 举起手臂
3. 向前走一步
4. 向前走一步
5. 向前走一步
6. 向右转 90 度
7. 向右转 90 度
8. 向前走一步
9. 向前走一步
10. 向前走一步
11. 向右转 90 度
12. 向右转 90 度
13. 放下手臂
14. 坐下

计算机解决问题的特点

- 只有若干可执行（操作）的指令
- 按需要将这些指令排列成一个序列（程序）
- 严格按照序列（程序）流程一步一步的执行

利用计算机处理问题的顺序

1. 分析问题
   只有对问题有一个清晰、准确的认定，才能够对问题求解。
2. 设计算法
   算法是求解问题方法及过程的形式化描述。
3. 编写程序
   使用计算机求解问题就要编写计算机程序，而程序都是根据算法来编写的。

什么是算法?

交换酱油和醋两瓶调料

有两瓶调料，分别标明的是“酱油”和“醋”。但由于工人疏忽，将瓶子灌装错了，现在要求将其交换回来。

设目前“酱油”瓶为 1 号瓶，它里面装的实际是醋；设目前“醋”瓶为 2 号瓶，它里面装的实际是酱油；设空瓶为 3 号瓶。

算法描述：

1. 将 1 号瓶中的醋倒入 3 号瓶
2. 将 2 号瓶中的酱油倒入 1 号瓶
3. 将 3 号瓶中的醋倒入 2 号瓶

通俗地讲，算法是步骤和描述

算法

算法 (Algorithm) 是指解题方案的准确而完整的描述，是一系列解决问题的清晰指令，算法代表着用系统的方法描述解决问题的策略机制。也就是说，能够对一定规范的输入，在有限时间内获得所要求的输出。

算法已经被广泛应用于计算机问题的求解。算法是求解问题方法及过程的形式化描述，是方法和步骤，是对具体问题求解的方法，是在计算机求解问题过程中的确定的、具体的、有限的步骤。

算法

算法表达了解决问题的核心步骤，反映的是程序的解题逻辑。算法其实并不是随着计算机的发明才出现的东西。

例如，早在两千多年前，古希腊数学家欧几里德就发明了一种求两个自然数的最大公约数的过程，这个过程被认为是史上第一个算法：

欧几里德算法：求自然数 a、b 的最大公约数

第 1 步：令 r 为 a / b 所得余数

第 2 步：若 r = 0，则算法结束，b 即为答案；否则置 a←b，b←r，转到第 1 步。

算法的应用

利用计算机解决问题的关键就在于设计出合适的算法，当今计算机在各行各业中的成功应用从根本上说都取决于高效算法的发现。

- 最成功的搜索引擎 Google：网页相关性排名算法
- 音频视频压缩算法节省存储空间
- GPS 导航仪利用高效的最短路径算法来规划最短路线
- 最成功的短视频 App 抖音：推荐算法

算法的分类

- **数值运算算法**
  主要用于解决数值求解问题。
  例如求方程的根、求函数的定积分、求最大公约数等。
- **非数值运算算法**
  主要用于解决需要用逻辑推理才能解决的问题。
  常见的是用于事务管理领域，例如图书检索、人事管理、行车调度管理、人机围棋大战、定制服务等。

算法的特征

- **确定性**
  算法的每个步骤必须有明确的含义，不会出现二义性。
- **有穷性**
  算法必须是由有限个步骤组成的过程，步骤的数量可能是几个，也可能是几百万个，但是必须有一个确定的结束条件。
- **可行性**
  算法的每一步都必须是可行的。
- **有零个或多个输入**
  所谓零个输入是指算法本身定出了初始条件。
- **有一个或多个输出**
  算法总是要解决特定的问题，问题来源就是算法的输入，期望的结果就是算法的输出。没有输入的算法是没有意义的，没有输出的算法是没有用的。

3. 算法

Section 3.1

算法的表示方法

算法的表示方法

一个问题可以设计不同的算法来求解，而同一个算法可以采用不同的形式来表述，常用的描述方法有以下几种：

- 自然语言
- 传统流程图
- 伪代码（类语言）
- 高级程序语言

自然语言

自然语言表示算法就是把算法的各个步骤，依次用人们熟悉的自然语言表示出来。日常生活中使用自然语言，通俗易懂，容易掌握，但是语句繁琐、文字冗长，很难清楚表达算法的逻辑流程，不严谨，容易出现“歧义性”，因此，自然语言通常用于介绍算法的概要思路。

“去市场买一个西瓜，如果看到西红柿，就买两个”

用自然语言描述：输入 x、y，计算 z=x÷y，输出 z。

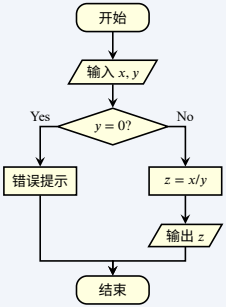
（1）输入变量 x，y；
（2）判断 y 是否为 0；
（3）如果 y=0，则输出出错提示信息；
（4）否则计算 z=x/y；
（5）输出 z。

传统流程图

流程图是描述算法的常用工具，是采用美国国家标准化协会规定的一组图形符号及文字说明来描述计算过程的图形。流程图直观地表示了算法的整个结构，易于理解，便于检查修改，但不方便表达数据处理过程。

图示	名称	说明
	开始/结束框	算法的开始或结束
	输入/输出框	变量的输入或输出
	处理框	变量的计算与赋值
	判断框	条件判断
→	流程线	算法的流向
○	连接点	算法的连接

输入 x、y，计算 z=x÷y，输出 z。



常用的制作流程图的软件

- **Visio**
  微软推出的一款流程图制作工具，内置大量的模板方便使用，界面简洁操作方便，功能全面。
- **Word/PowerPoint**
  可以做基本的流程图，容易上手。
- **网页版在线作图工具**
  - Processon
  - draw.io

伪代码

伪代码也称类语言，接近于高级语言但又不是严格意义上的高级语言，是一种介于自然语言与计算机语言之间的算法描述方式。伪代码具有高级语言的一般语句结构，撇掉语言中的细节，以便把注意力主要集中在算法处理步骤本身的描述上。

算法 Calculate $y = x/y$

```
begin
input x, y;
if y=0 then
    print "Error!";
else
    z ← x/y
return(z);
end
```

高级程序语言

高级程序语言就是用计算机程序设计语言直接表达算法，是可以在计算机上直接运行的源程序。高级程序语言描述的算法具有严格、准确的优点，但也有语言细节过多的弱点。

用 python 语言描述：输入 x、y，计算 z

```python
x = float(input("请输入x: "))
y = float(input("请输入y: "))
if y == 0:
    print("分母不能为0! ")
else:
    z = x/y
    print(z)
```

Section 3.2

算法的结构

算法的三种结构

- **顺序结构**

  这种结构最简单，各个步骤是按先后顺序执行的。

- **分支结构**

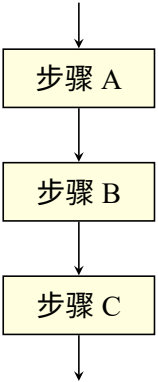
  分支结构又称选择结构，用于判断给定的条件，根据判断的结果判断某些条件，根据判断的结果来控制程序的流程。

- **循环结构**

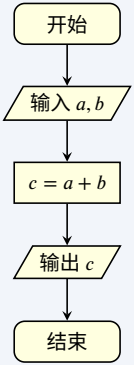
  循环结构又称为重复结构，指在程序中需要反复执行某个功能而设置的一种程序结构。它由循环体中的条件，判断继续执行某个功能还是退出循环。

顺序结构

顺序结构的各个步骤是按先后顺序执行的。



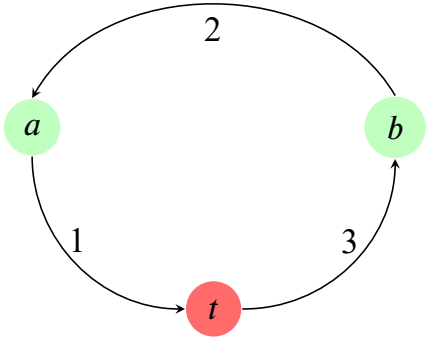
输入 a 和 b，输出 a+b 的和

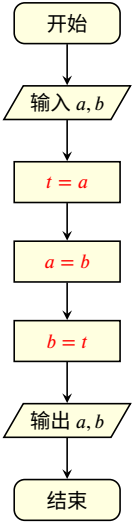


顺序结构

例

交换 2 个变量的值，然后输出。

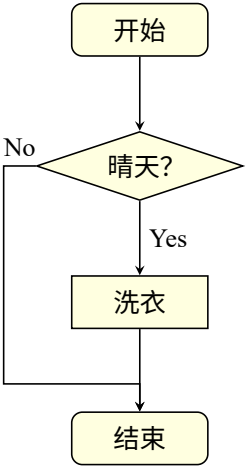




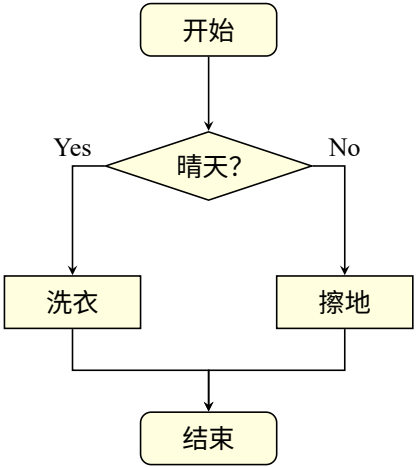
分支结构

有些操作需要满足一定的条件才能进行，在操作前需要进行条件判断。

单分支：



双分支：

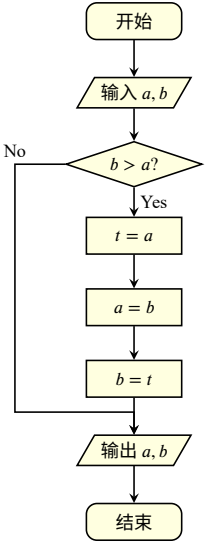


分支结构

例

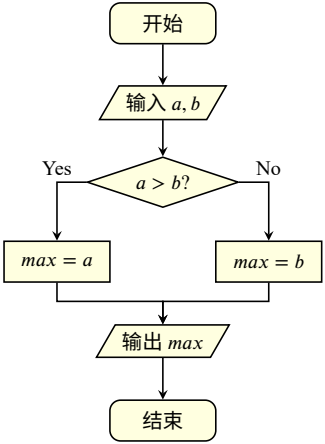
将 a，b 变量中的数据按从大到小的顺序排序，即处理后 a 中存放两数中的大数，b 存放小者。

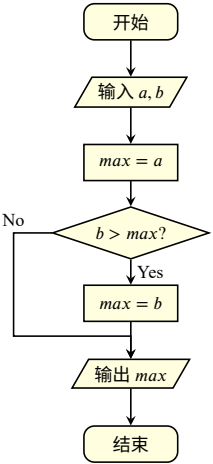
- 如果 a>b，则不需要处理；
- 如果 a<b，则需要交换 a 与 b 的值。



例

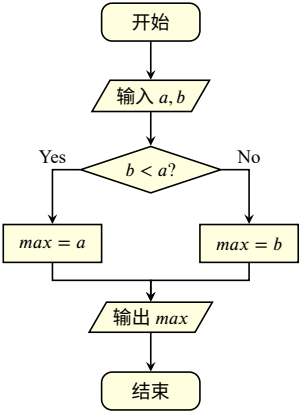
输入 2 个数，求最大值输出。

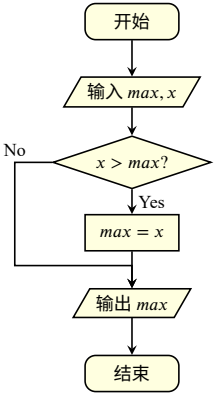


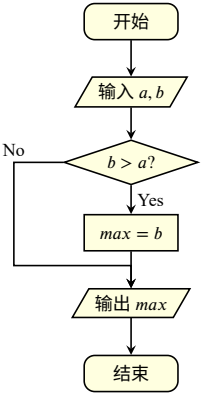


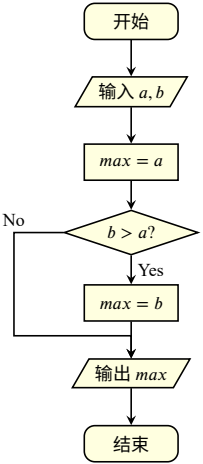
例

下列求 2 个数的最大值的流程图中哪个是错误的?



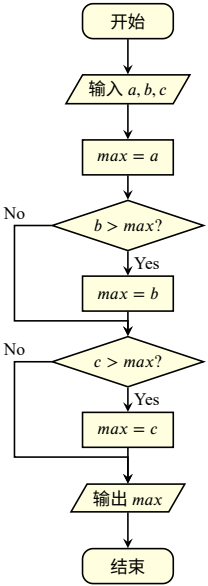






例

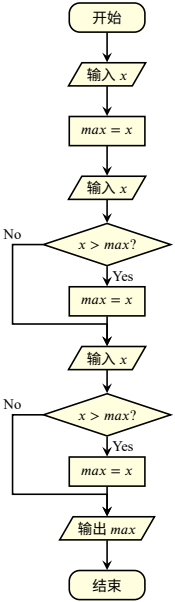
输入 3 个数，求最大值输出



例

输入 3 个数，求最大值输出

实际上，在求最大值时不需要一直保存输入的 3 个数，输入的数据可先后存放到变量 x 中，每次输入一个新的数据时，上一次输入的数据就被覆盖掉。



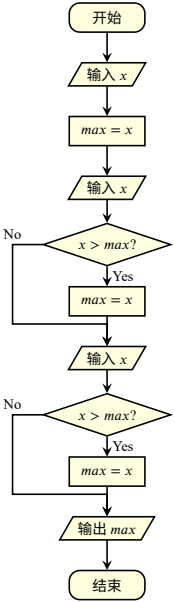
例

输入 3 个数，求最大值输出

实际上，在求最大值时不需要一直保存输入的 3 个数，输入的数据可先后存放到变量 x 中，每次输入一个新的数据时，上一次输入的数据就被覆盖掉。

思考

那么如何求 4 个、5 个数的最大值呢？



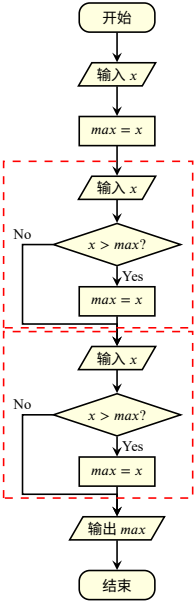
例

输入 3 个数，求最大值输出

实际上，在求最大值时不需要一直保存输入的 3 个数，输入的数据可先后存放到变量 x 中，每次输入一个新的数据时，上一次输入的数据就被覆盖掉。

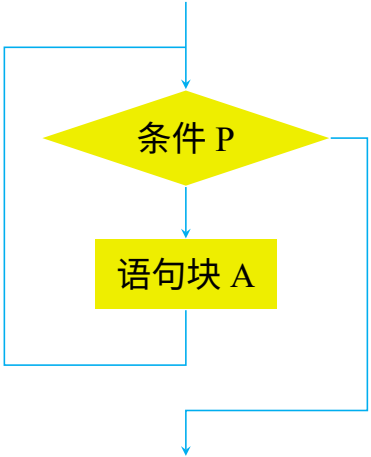
思考

那么如何求 4 个、5 个数的最大值呢？



循环结构

在解决问题时，并不是所有操作只做 1 次，有些操作要重复操作多次，称为循环。



循环的组成：

A 循环体

P 循环控制条件

循环结构的设计：

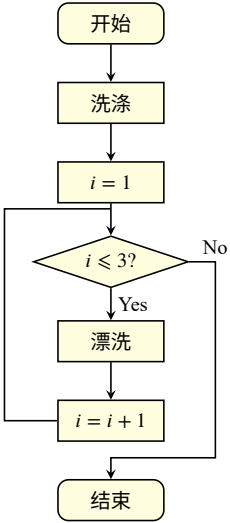
- 循环次数已知
- 循环次数未知

循环结构

循环次数已知的情况

人在进行重复几次的操作时，心里肯定会进行计数，可称为 **计数器**。

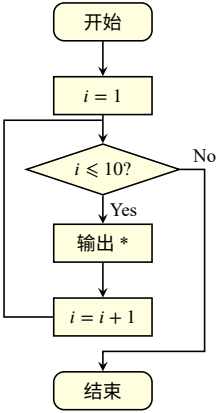
计数器的使用：
初值为 1；每循环一次，加 1；每次循环前查看它是否超过规定的次数。没有超过，继续循环，超过了则循环结束。
例如要进行 3 次漂洗。算法中设置 1 个变量存放计数器的值，如图中的变量 i。



循环结构

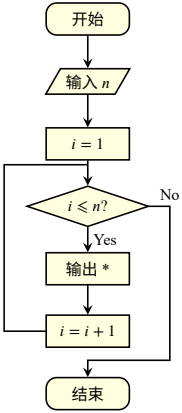
例

输出 10 个星号 *。



例

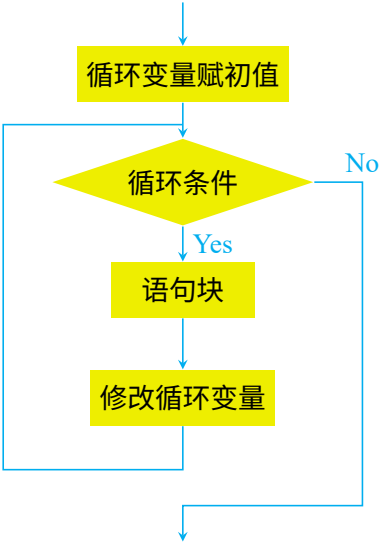
输出 n 个星号 *。



循环结构

在构造这种循环结构时，应注意以下 3 点：

1. 为循环变量赋初值
2. 设置循环条件
3. 修改循环变量的值



循环结构

循环次数未知的情况

有些情况下，循环的次数无法确定，此时，则需要设计一个条件来控制循环。

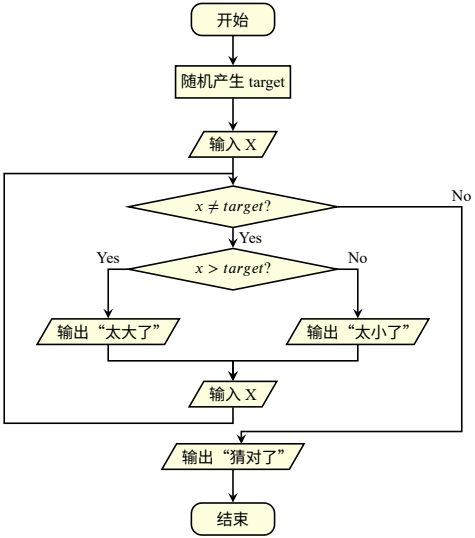
例 猜数字。

计算机随机产生一个 100 以内的整数 target，请用户猜。

若猜对了即可结束，并输出“猜对了”，否则继续猜。

这里猜的次数无法确定，

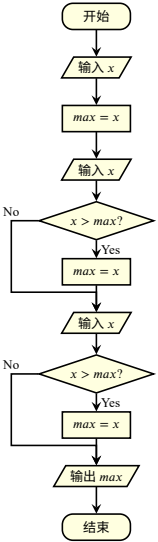
循环条件：未猜对



循环结构

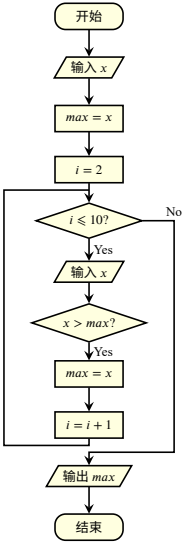
分支结构

求 3 个数中的最大值



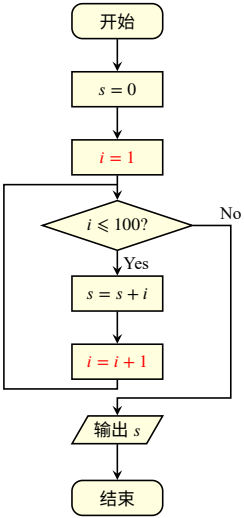
循环结构

求 10 个数中的最大值

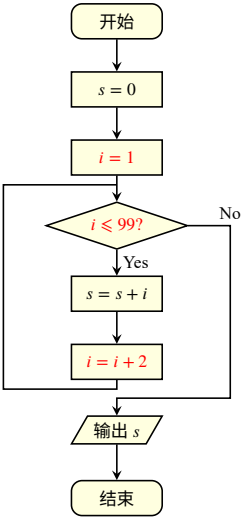


循环结构-求和

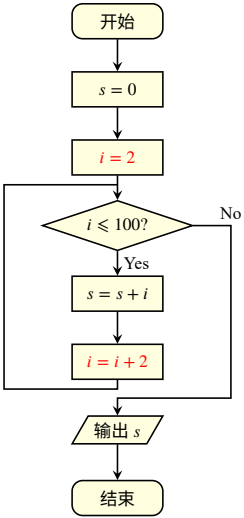
$1+2+3+\cdots+100$



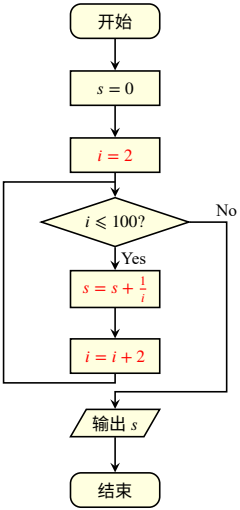
$1+3+5+\cdots+99$



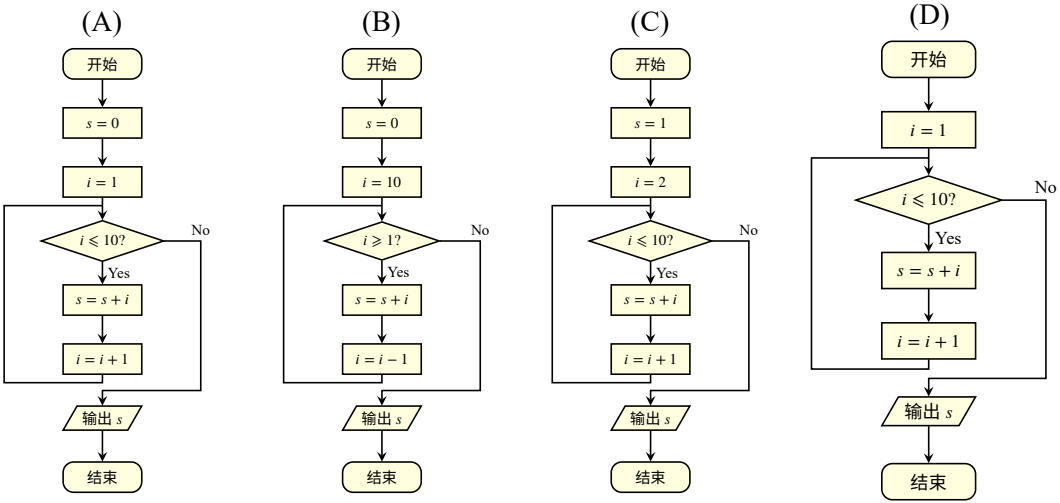
$2+4+6+\cdots+100$



$\frac{1}{2}+\frac{1}{4}+\frac{1}{6}+\cdots+\frac{1}{100}$



下列是求 1 到 10 这 10 个整数的和的流程图，哪个是错误的?

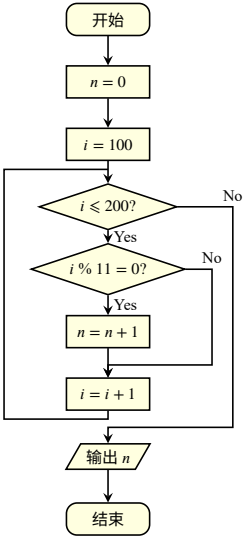


循环结构-计数

例

统计 100 到 200 之间能够被 11 整除的数的个数。

1. 设置 n 用于计数，并赋初值为 0；
2. 设置 i 用于控制循环次数，初值为 100；循环条件为 $i \leqslant 200$?
3. 在循环中对 i 除 11 的余数是否为 0 进行判断，若为 0，则 $n = n + 1$，即进行计数；
4. 循环完毕，即可统计出符合要求数据的个数。

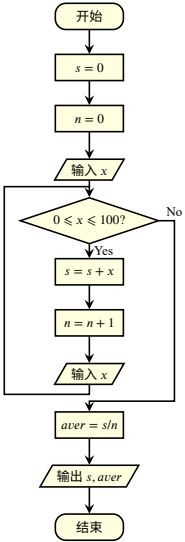


循环结构

例

输入若干学生的百分制成绩，计算总分和平均分。
若输入的成绩不是百分制成绩，则表示输入结束。

1. 设置 s 用于求和，初值设为 0；
2. 设置 n 用于计数，初值设为 0；
3. 设置 x 用于接收输入的成绩；
4. 设置循环条件，$0 \leqslant x \leqslant 100$?

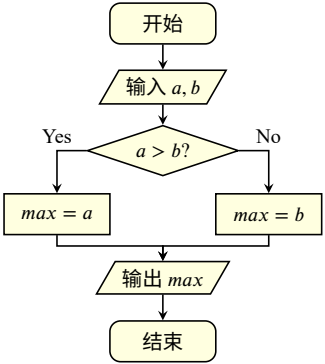


Section 3.3

从算法到程序

算法的实现

给定一个问题，当我们找到解决问题的算法后，接着就需要用某种计算机语言将这个算法表达出来，最终得到一个能被计算机执行的程序（或代码），这个过程称为实现（implementation），或者俗称为写代码（coding）。



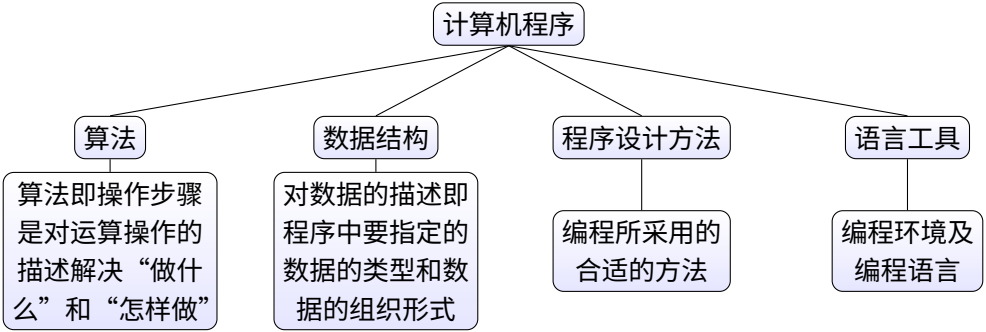
输入 2 个整数，输出其中最大值

```python
a = input("请输入a: ")
b = input("请输入b: ")
a = int(a)
b = int(b)
if b < a:
    max = a
else:
    max = b
print(max)
```

计算机程序

算法描述了问题处理的方式或步骤，程序则是采用具体语言规则实现了算法的功能，算法要依靠程序来完成功能，算法是程序的灵魂。

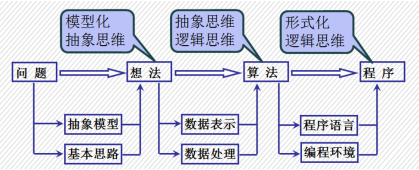
程序是计算机解决实际问题的计算步骤。它是有始与有终的闭环，有选择，有循环，有过程，有函数等，算法是程序的核心。



程序设计的一般过程

程序设计是借助一定工具（编程环境）利用计算机解决实际问题的全过程。包括建立模型，数据组织，设计算法，程序实现，上机调试，结果反馈及算法分析的全过程。可以用**三阶段六步骤**来进行概括。

1. 问题到想法
2. 想法到算法
3. 算法到程序

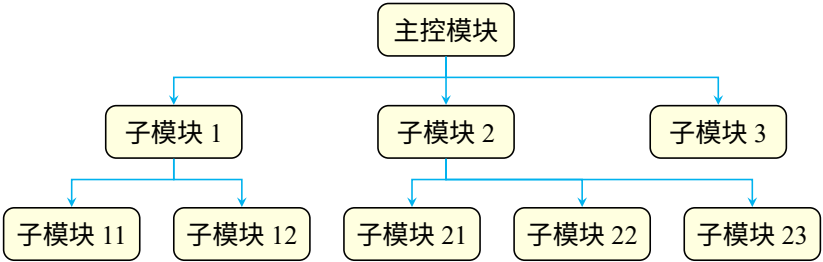


1. 界定与分析问题
2. 建模
3. 选择解决方法
4. 设计处理算法
5. 程序化
6. 算法分析

程序的控制结构

结构化的程序设计就是把一个程序分成若干互相独立的模块。基本要点：

1. 自上而下，逐步细化
   将复杂的大问题逐步分解多个简单的小问题。
2. 模块化设计
   一个程序是由一个主控模块和若干个子模块组成的。
3. 结构化编码



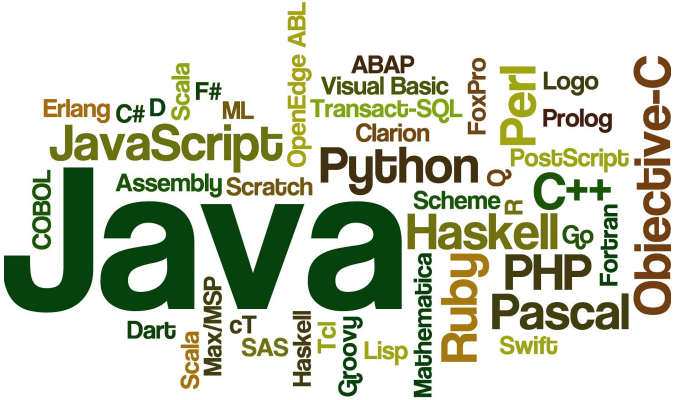
编程语言

和人沟通：

- 汉语
- 英语
- ……

和计算机沟通：

- C
- python
- java
- ……



编程语言流行度

图: TIOBE Index for November 2023

Nov 2023	Nov 2022	Change	Programming Language	Ratings	Change
1	1		Python	14.16%	-3.02%
2	2		C	11.77%	-3.31%
3	4	^	C++	10.36%	-0.39%
4	3	v	Java	8.35%	-3.63%
5	5		C#	7.65%	+3.40%
6	7	^	JavaScript	3.21%	+0.47%
7	10	^	PHP	2.30%	+0.61%
8	6	v	Visual Basic	2.10%	-2.01%
9	9		SQL	1.88%	+0.07%
10	8	v	Assembly language	1.35%	-0.83%

计算机二级

- C 语言程序设计
- Java 语言程序设计
- C++ 语言程序设计
- Python 语言程序设计
- Web 程序设计
- MS Office 高级应用与设计
- WPS Office 高级应用与设计
- Access 数据库程序设计
- MySQL 数据库程序设计
- openGauss 数据库程序设计

编程语言对比

	优点	缺点	典型应用领域
Python	易于学习 丰富的库	运行速度较慢 内存占用较大	科学计算和数据分析 机器学习和人工智能
C	运行速度快 跨平台性非常好	复杂的学习曲线 不支持面向对象编程	操作系统开发 嵌入式系统
Java	自动管理内存 跨平台性非常好	占用大量内存 启动时间较长	Web 编程 应用程序开发
C++	性能高 可以直接访问硬件	学习曲线陡峭 代码可读性差	游戏开发 操作系统和驱动程序

学习程序设计的好处

1. 计算机已经成为我们生活、学习和工作中普遍使用的工具，学会编程能使我们解决很多问题。
2. 程序设计能够培养我们抽象、分析和问题求解的能力，这种能力对日常生活和工作也是很重要的。
3. 编程也是一种充满乐趣的智力活动，许多人将编程当作一种爱好，发现巧妙的算法和程序运行成功后的那种成就感令人乐此不疲。

随机点名程序

```python
import random
import tkinter as tk


class RollCallApp:
    def __init__(self, students):
        self.students = students
        self.roll_call_running = False

        self.window = tk.Tk()
        self.window.title("班级点名程序")
        self.window.geometry("400x200")

        self.student_name = tk.StringVar()
        self.student_name.set("请开始点名")
```

```python

        self.label = tk.Label(self.window, textvariable=self.student_name, font=("微软雅黑", 24), fg="red")
        self.label.pack(pady=20)

        self.button = tk.Button(self.window, text="开始点名", font=("微软雅黑", 16), command=self.roll_call)
        self.button.pack(pady=10)

    def roll_call(self):
        if not self.roll_call_running:
            self.roll_call_running = True
            self.button.config(text="停止滚动")
            self.roll_call_loop()
        else:
            self.roll_call_running = False
            self.button.config(text="开始点名")
```

```python

    def roll_call_loop(self):
        student = random.choice(self.students)
        self.student_name.set(student)

        if self.roll_call_running:
            self.window.after(100, self.roll_call_loop)

    def run(self):
        self.window.mainloop()


students = ["Alice", "Bob", "Charlie", "David", "Emily", "Frank", "Grace", "Harry", "
    Isabella", "Jack", "Kevin", "Lily", "Maggie", "Nancy", "Oliver", "Peter"]

app = RollCallApp(students)
app.run()
```

Section 3.4

算法的评价

算法的评价

解决某一问题存在多种算法，为了有效地进行解题，不仅需要保证算法正确，还要考虑算法的质量，那怎样才算是一个优秀的算法呢？一般有这么几个标准：

（1）正确性（2）可读性（3）健壮性（4）效率

以效率为标准举例：称金币问题

9 个外观一样的金币，其中一个赝品重量较轻。如果用天平秤鉴别真伪，至少需要称几次才能保证找出假币？

算法的评价

解决某一问题存在多种算法，为了有效地进行解题，不仅需要保证算法正确，还要考虑算法的质量，那怎样才算是一个优秀的算法呢？一般有这么几个标准：

（1）正确性（2）可读性（3）健壮性（4）效率

以效率为标准举例：称金币问题

9 个外观一样的金币，其中一个赝品重量较轻。如果用天平秤鉴别真伪，至少需要称几次才能保证找出假币？

算法 1：天平左边金币固定，不断变换右边金币，最多称 7 次可鉴别出假币。

算法 2：天平两边各一个金币，每次变换两边金币，最多称 4 次可鉴别出假币。

算法 3：天平左边 3 个，右边 3 个，留下 3 个，最多称 2 次可以鉴别出假币。

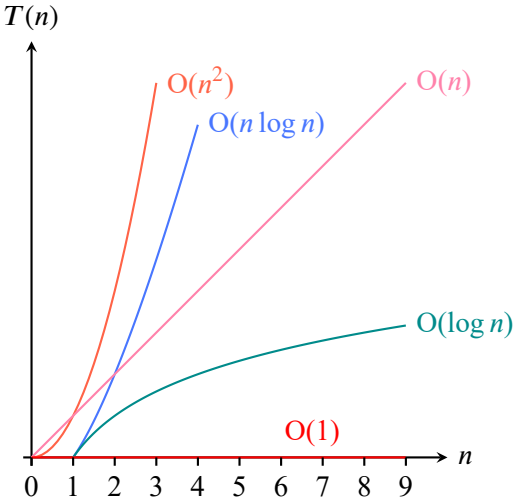
算法复杂度

为了更公平的比较和衡量算法质量的优劣程序，需要一个客观的标准，这个标准就是著名计算机科学家，算法分析之父高德纳提出的**算法复杂度**。

- **时间复杂度**
  有许多因素可能会影响算法在特定计算机上的运行时间，因此，大 O 表示法不关注算法运行的实际时间，而是根据需要执行的操作数量来定义运行时间。
- **空间复杂度**
  该算法所耗费的存储空间
  在平时的开发中更多的是用空间换时间

常见时间复杂度

大 O 记法	非正式术语
$O(n^2)$	平方阶
$O(n \log n)$	线性对数阶
$O(n)$	线性阶
$O(\log n)$	对数阶
$O(1)$	常数阶



4. 排序算法

Section 4.1

选择排序

排序算法

「排序算法 sorting algorithm」用于对一组数据按照特定顺序进行排列。排序算法有着广泛的应用，因为有序数据通常能够被更有效地查找、分析和处理。如图所示，排序算法中的数据类型可以是整数、浮点数、字符或字符串等。

按照数字大小排序整数

8	3	2	6	7	1	5	4

→

1	2	3	4	5	6	7	8

按照字典顺序排序字符串

Karl	Mike	Alice	Bob

→

Alice	Bob	Karl	Mike

选择排序

选择排序的工作原理：通过一轮循环找到最小值，把它放到第一个位置，然后在剩余的数据中再找到最小值，放到第二个位置，以此类推，直到所有值排序完成。

1. 初始状态下，所有元素未排序，即未排序（索引）区间为 $[0, n-1]$ 。
2. 选取区间 $[0, n-1]$ 中的最小元素，将其与索引 0 处元素交换。完成后，数组前 1 个元素已排序。
3. 选取区间 $[1, n-1]$ 中的最小元素，将其与索引 1 处元素交换。完成后，数组前 2 个元素已排序。
4. 以此类推。经过 $n-1$ 轮选择与交换后，数组前 $n-1$ 个元素已排序。
5. 仅剩的一个元素必定是最大元素，无须排序，因此数组排序完成。

图解

8	3	2	6	7	1	5	4

第 1 轮排序

1	8	3	2	6	7	5	4

第 2 轮排序

1	2	8	3	6	7	5	4

第 3 轮排序

1	2	3	8	6	7	5	4

第 4 轮排序

1	2	3	4	8	6	7	5

第 5 轮排序

1	2	3	4	5	8	6	7

第 6 轮排序

1	2	3	4	5	6	8	7

第 7 轮排序

1	2	3	4	5	6	7	8

代码

选择排序

```python
def selection_sort(nums: list[int]):
    """ 选择排序"""
    n = len(nums)
    # 外循环：未排序区间为 [i, n-1]
    for i in range(n - 1):
        # 内循环：找到未排序区间内的最小元素
        k = i
        for j in range(i + 1, n):
            if nums[j] < nums[k]:
                k = j
                # 记录最小元素的索引
                # 将该最小元素与未排序区间的首个元素交换
        nums[i], nums[k] = nums[k], nums[i]
```

算法时间复杂度

- 比较次数：$(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}$
- 交换次数：$n-1$
- 总次数：$\frac{(n+2)(n-1)}{2}$

时间复杂度：$O(n^2)$

Section 4.2

冒泡排序

冒泡排序

冒泡排序把一组数据从左边开始进行两两比较，小的放前面，大的放后面，反复比较直到没有数据需要交换为止。

1. 从队首开始，两两比较数值，把大的往后交换，一直到队尾，此时数组的最大元素交换至正确位置；
2. 从队首开始，对剩余 $n-1$ 个元素执行“冒泡”，将第二大元素交换至正确位置；
3. 以此类推，经过 $n-1$ 轮“冒泡”后，前 $n-1$ 大的元素都被交换至正确位置。
4. 仅剩的一个元素必定是最小元素，无须排序，因此数组排序完成。

图解

8	3	2	6	7	1	5	4

第 1 轮冒泡

3	2	6	7	1	5	4	8

第 2 轮冒泡

2	3	6	1	5	4	7	8

第 3 轮冒泡

2	3	1	5	4	6	7	8

第 4 轮冒泡

2	1	3	4	5	6	7	8

第 5 轮冒泡

1	2	3	4	5	6	7	8

第 6 轮冒泡

1	2	3	4	5	6	7	8

第 7 轮冒泡

1	2	3	4	5	6	7	8

代码

冒泡排序

```python
def bubble_sort(nums: list[int]):
    """ 冒泡排序"""
    n = len(nums)
    # 外循环：未排序区间为 [0, i]
    for i in range(n - 1, 0, -1):
        # 内循环：将未排序区间 [0, i] 中的最大元素交换至该区间的最右端
        for j in range(i):
            if nums[j] > nums[j + 1]:
                # 交换 nums[j] 与 nums[j + 1]
                nums[j], nums[j + 1] = nums[j + 1], nums[j]
```

冒泡排序效率优化

如果某轮“冒泡”中没有执行任何交换操作，说明数组已经完成排序，可直接返回结果。因此，可以增加一个标志位 flag 来监测这种情况，一旦出现就立即返回。

```python
def bubble_sort(nums: list[int]):
    """ 冒泡排序"""
    n = len(nums)
    for i in range(n - 1, 0, -1):
        flag = False # 初始化标志位
        for j in range(i):
            if nums[j] > nums[j + 1]:
                # 交换 nums[j] 与 nums[j + 1]
                nums[j], nums[j + 1] = nums[j + 1], nums[j]
                flag = True # 记录交换元素
        if not flag:
            break # 此轮冒泡未交换任何元素，直接跳出
```

算法时间复杂度

- 冒泡次数：$(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}$

冒泡排序的最差和平均时间复杂度仍为：$O(n^2)$

拿数组 [1,2,3,4,5,6] 为例，这个比较极端完全是有序的，只需要 1 次冒泡就完成了。在完全有序的情况下，最好的时间复杂度是 O(n)。

Section 4.3

插入排序

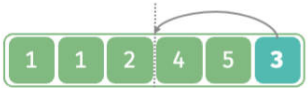
插入排序

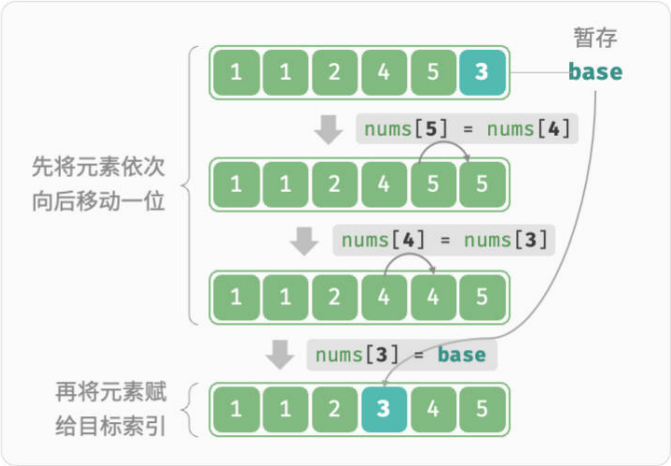
插入排序的工作原理与手动整理一副牌的过程非常相似。具体来说，我们在未排序区间选择一个基准元素，将该元素与其左侧已排序区间的元素逐一比较大小，并将该元素插入到正确的位置。

1. 初始状态下，数组的第 1 个元素已完成排序。
2. 选取数组的第 2 个元素作为 base ，将其插入到正确位置后，数组的前 2 个元素已排序。
3. 选取第 3 个元素作为 base ，将其插入到正确位置后，数组的前 3 个元素已排序。
4. 以此类推，在最后一轮中，选取最后一个元素作为 base ，将其插入到正确位置后，所有元素均已排序。

单次插入操作

将元素 **3** 插入到 **nums**
已排序区间的正确位置





图解

8	3	2	6	7	1	5	4

第 1 轮插入

3	8	2	6	7	1	5	4

第 2 轮插入

2	3	8	6	7	1	5	4

第 3 轮插入

2	3	6	8	7	1	5	4

第 4 轮插入

2	3	6	7	8	1	5	4

第 5 轮插入

1	2	3	6	7	8	5	4

第 6 轮插入

1	2	3	5	6	7	8	4

第 7 轮插入

1	2	3	4	5	6	7	8

代码

插入排序

```python
def insertion_sort(nums: list[int]):
    """ 插入排序"""
    # 外循环：已排序区间为 [0, i-1]
    for i in range(1, len(nums)):
        base = nums[i]
        j = i - 1
        # 内循环：将 base 插入到已排序区间 [0, i-1] 中的正确位置
        while j >= 0 and nums[j] > base:
            nums[j + 1] = nums[j] # 将 nums[j] 向右移动一位
            j -= 1
        nums[j + 1] = base # 将 base 赋值到正确位置
```

算法时间复杂度

- 最差情况下，每次插入操作分别需要循环 $n-1$、$n-2$、…、2、1 次，求和得到 $\frac{n(n-1)}{2}$，因此时间复杂度为 $O(n^2)$
- 在遇到有序数据时，插入操作会提前终止。当输入数组完全有序时，插入排序达到最佳时间复杂度 $O(n)$
- 如果排序记录是随机的，根据概率相同的原则，平均比较和移动的次数约为 $n^2/4$ 次，比冒泡排序和简单选择排序算法性能好一些。

虽然冒泡排序、选择排序和插入排序的时间复杂度都为 $O(n^2)$，但在实际情况中，插入排序的使用频率显著高于冒泡排序和选择排序。

Section 4.4

快速排序

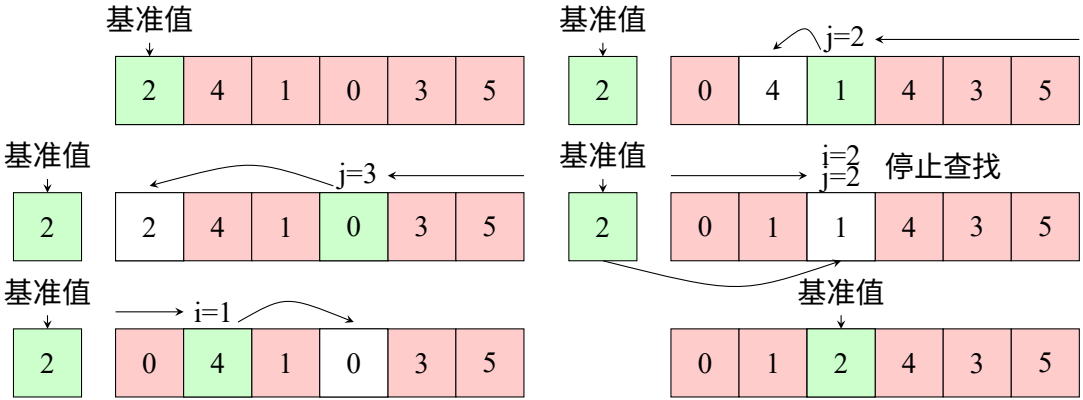
快速排序

快速排序是一种基于分治策略的排序算法，运行高效，应用广泛。

快速排序的核心思想是：选择数组中的某个元素作为“基准数”（哨兵元素），通过一轮排序将要排序的数据分割成独立的两部分，所有小于基准数的元素移到其左侧，而大于基准数的元素移到其右侧。这个基准数可以任取一个，一般取开始、结束或中间位置的一个元素。

1. 选取列表第一个数值作为基准数 P，把比 P 小的数放前面，把比 P 大的数放后面。
2. 分别对前面部分和后面部分的数据按照 (1) 的原则进行排列。
3. 一直到每一部分只剩一个元素，完成快速排序。

按基准数排序


基准值
2 4 1 0 3 5
基准值
2
j=3
2 4 1 0 3 5
基准值
2
i=1
0 4 1 0 3 5
基准值
2
j=2
0 4 1 4 3 5
基准值
2
i=2
j=2
停止查找
0 1 1 4 3 5
基准值
0 1 2 4 3 5

图解

8	3	2	6	7	1	5	4

第 1 轮快排

4	3	2	6	7	1	5	8

第 2 轮快排

1	3	2	4	7	6	5	8

第 3 轮快排

1	3	2	4	5	6	7	8

第 4 轮快排

1	2	3	4	5	6	7	8

代码

快速排序 (1)

```
def quick_sort(alist, start, end):
    if start >= end: # 递归的退出条件
        return
    mid = alist[start] # 设定起始的基准元素
    low = start # low为序列左边在开始位置的由左向右移动的游标
    high = end # high为序列右边末尾位置的由右向左移动的游标
    while low < high:
        # 如果low与high未重合，high(右边)指向的元素大于等于基准元素，则high向左移动
        while low < high and alist[high] >= mid:
            high -= 1
        alist[low] = alist[high]
        while low < high and alist[low] < mid:
            low += 1
        alist[high] = alist[low]
```

代码

快速排序 (2)

```
    # 退出循环后，low与high重合，此时所指位置为基准元素的正确位置,左边的元素都比基准元素小,右边的元素都比基准元素大
    alist[low] = mid # 将基准元素放到该位置,
    # 对基准元素左边的子序列进行快速排序
    quick_sort(alist, start, low - 1) # start :0 low -1 原基准元素靠左边一位
    # 对基准元素右边的子序列进行快速排序
    quick_sort(alist, low + 1, end) # low+1 : 原基准元素靠右一位 end: 最后


if __name__ == '__main__':
    alist = [54, 26, 93, 17, 77, 31, 44, 55, 20]
    quick_sort(alist, 0, len(alist) - 1)
    print(alist)
```

算法时间复杂度

- 在平均情况下，递归层数为 $\log n$ ，每层中的总循环数为 n，总体使用 $O(n \log n)$ 时间。
- 在最差情况下，每轮操作都将长度为 n 的数组划分为长度为 0 和 $n-1$ 的两个子数组，此时递归层数达到 n 层，每层中的循环数为 n，总体使用 $O(n^2)$ 时间。
- 出现最差情况的概率很低，在绝大多数情况下，快速排序能在 $O(n \log n)$ 的时间复杂度下运行。

基准数优化

快速排序在某些输入下的时间效率可能降低。举一个极端例子，假设输入数组是完全倒序的，由于我们选择最左端元素作为基准数，那么在哨兵划分完成后，基准数被交换至数组最右端，导致左子数组长度为 $n-1$、右子数组长度为 0。如此递归下去，每轮哨兵划分后的右子数组长度都为 0，分治策略失效，快速排序退化为“冒泡排序”。

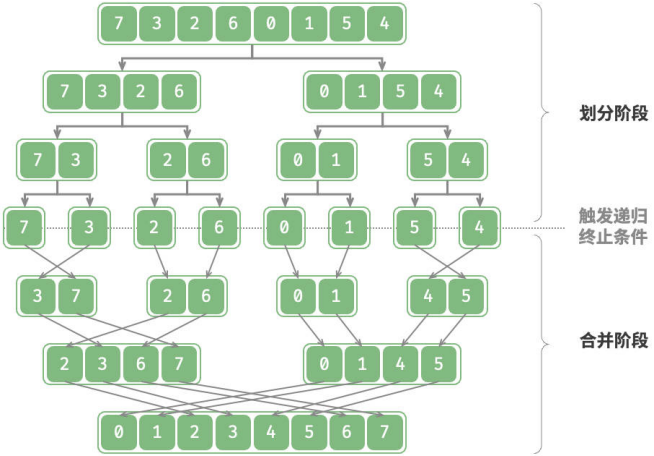
为了尽量避免这种情况发生，我们可以优化基准数的选取策略。我们可以在数组中选取三个候选元素（通常为数组的首、尾、中点元素），并将这三个候选元素的中位数作为基准数。这样一来，基准数“既不太小也不太大”的概率将大幅提升。当然，我们还可以选取更多候选元素，以进一步提高算法的稳健性。采用这种方法后，时间复杂度劣化至 $O(n^2)$ 的概率大大降低。

Section 4.5

归并排序

归并排序

归并排序是一种基于分治策略的排序算法，包含“划分”和“合并”阶段。



算法时间复杂度

- 划分为 $\log n$ 层
- 每层合并的总操作数量为 n
- 总体时间复杂度为 $O(n \log n)$

Section 4.6

奇葩排序

猴子排序

将无限只猴子置于无限台打字机前，等待无限长的时间，那么猴子也能完整地写出一部《哈姆雷特》。

—波莱尔的无限猴子实验

真猴子实验

2003 年，英国一所大学的师生给动物园的猴子买了一台电脑和一个键盘。在一个月的时间里，猴子除了胡乱敲键盘外，剩下的时间便是在电脑上进行大小便，最终导致电脑无法正常运行，实验宣告失败。

程序模拟猴子

2004 年，电脑模拟的猴子在花费了$4.216\,25 \times 10^{28}$ 年后，终于打出了莎士比亚的剧作《维洛那二绅士》的前十九个字母 VALENTINE.Cease to。

猴子排序

基于上述理论，便可以得到猴子排序算法。对于要排序的数组，每一次给它进行一个随机的排序，那么总有一次，它能够变成一个有序的数组。

- 如果运气好，可能一次就搞定。一次成功的幸运猴，时间复杂度就是 $O(n)$
- 如果运气不好，可能一直都不会成功。永不成功猴，时间复杂度就是 $O(\infty)$

例

尝试用猴子排序算法对 10 个 $1 \sim 1000$ 范围内的数据做排序，看看最终要尝试多少次，花费多长时间？

睡眠排序

老板有毛病吧，写完排序就叫我走人，我还嫌你这9K工资低了呢

```java
public class ArraySort implements Runnable {
    private String num;

    public ArraySort(int num) {
        this.num = num + "";
    }

    public static void main(String[] args) {
        // 把这个数组升序输出
        int[] nums = { 11, 3, 998, 5455, 1, 152, 990 };
        for (int i = 0; i < nums.length; i++) {
            new Thread(new ArraySort(nums[i])).start();
        }

    }

    public void run() {
        try {
            Thread.sleep(Integer.parseInt(num));
            System.out.println(num);
        } catch (Exception e) {
            e.printStackTrace();
        }

    }
}
```

```
Console
<terminated> Ar
1
3
11
152
990
998
5455
```

这脑回路，简直是程序员界的一股清流啊，9K 说的是年薪吧

多高端呀，都用上多线程了

宝才，公司捡到鬼了

我的老天鹅呀！他真的能用，不过我并不想用 2147483637 秒去排 2147483647 个数字。。。

没有什么问题是睡一觉不能解决的，如果有，就多睡一会儿

睡眠排序

构造 n 个线程，它们和这 n 个数一一对应。初始化后，线程们开始睡眠，等到对应的数那么多个时间单位后各自醒来，然后输出它对应的数。这样最小的数对应的线程最早醒来，这个数最早被输出，数值越大的线程就会越晚醒来，等所有线程都醒来，排序就结束了。

- 睡眠排序的耗时取决于数组中最大的那个数字，数字越大，耗时越久；
- 当数组中存在负数时，运行就会报错，因为线程的睡眠时间不能为负数。
- 实际上，睡眠排序面临许多问题
  - 输入数据很相近时会有误差
  - 输入数据很多时，这些线程不能看作是同时启动
  - 输入数据太大时，运行时间太长

5. 常用算法

Section 5.1

枚举算法

枚举法的基本思想

根据题目的部分条件确定答案的大致范围，并在此范围内对所有可能的情况逐一验证，直到全部情况验证完毕。枚举法也称为穷举法。

子数整数问题

对于一个五位数 $a_1a_2a_3a_4a_5$，可将其拆分为三个子数：sub1、sub2 和 sub3。

$\text{sub1} = a_1a_2a_3$

$\text{sub2} = a_2a_3a_4$

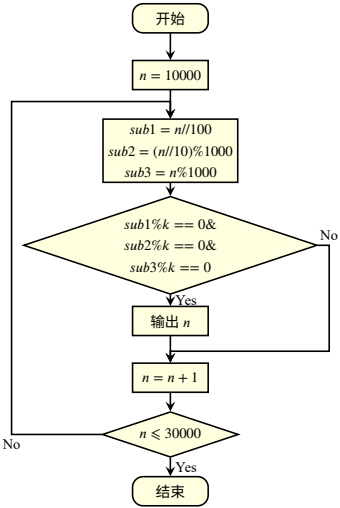
$\text{sub3} = a_3a_4a_5$

现在给定一个正整数 K，要求求出 10000 到 30000 之间所有满足下述条件的五位数，条件是这些五位数的三个子数 sub1，sub2，sub3 都可被 K 整除。

子数整数问题

算法

1. n=10000
2. 对 n 通过整除和求余运算得到子数
   - sub1=n//100
   - sub2=(n //10) % 1000
   - sub3=n % 1000
3. 判断 sub1、sub2、sub3 是否都可被 K 整除。
4. 如果满足，输出 n，n++；
5. 如果不能整除 k，n++；
6. If n<=30000，goto 步骤 2，else 结束。



子数整数问题 Python 代码

子数整数问题

```python
k = int(input("请输入k: "))
for value in range(10000, 30001):
    sub1 = value // 100
    sub2 = value // 10 % 1000
    sub3 = value % 1000
    if sub1 % k == 0 and sub2 % k == 0 and sub3 % k == 0:
        print(value)
```

枚举算法的特点

- 枚举法是一种几乎什么问题都能解决的一般性方法。
- 简单粗暴，他暴力的枚举所有可能。
- 效率低下，但是可用枚举法求解一些小规模的问题实例。
- 如果解决问题实例不多，而枚举法可用一种可接受的速度对问题求解，那么花时间去设计一个更高效的算法是得不偿失的。

Section 5.2

迭代算法

迭代算法

迭代法也称为“辗转法”，是一种不断用变量的旧值推算出新值的方法。

迭代算法是用计算机解决问题的一种基本方法，它利用计算机运算速度快、适合做重复性操作的特点，让计算机对一组指令 (或一定步骤) 进行重复执行，在每次执行这组指令 (或这些步骤) 时，都从变量的原值推出它的一个新值。

求 1 到 100 的和

```
i = 0
sum = 0
while i < 101:
    sum = sum + i
    i = i + 1
```

迭代算法验证谷角猜想

日本数学家谷角静夫在研究自然数时发现了一个奇怪现象：对于任意一个自然数 n，若 n 为偶数，则将其除以 2；若 n 为奇数，则将其乘以 3，然后再加 1。如此经过有限次运算后，总可以得到自然数 1。

算法分析：

1. 确定迭代变量，n 就是迭代变量；
2. 确定迭代关系，有两种迭代关系：一种是当 n 为偶数时 n=n/2，一种是当 n 为奇数时 n=n*3+1；
3. 迭代控制，通常就是找到循环结束条件，本例就是当 n=1 结束，验证猜想成功。

迭代法验证谷角猜想

```python
n = int(input("请输入n: "))
while n != 1:
    if n % 2 == 0:
        n = n / 2
    else:
        n = 3 * n + 1
print("猜想成功")
```

牛顿迭代法求平方根

已知求 a 的平方根的迭代公式为：$x_{n+1} = \frac{1}{2}(x_n + \frac{a}{x_n})$

牛顿迭代法求平方根

```python
a = float(input("请输入a: "))
x0 = float(input("请输入x0: "))
x1 = 0.5 * (x0 + a / x0)
while abs(x1 - x0) > 0.000001:
    x0 = x1
    x1 = 0.5 * (x0 + a / x0)
print(x1)
```

Section 5.3

分治算法

分治算法

分治，顾名思义，分而治之。

分治法（divide and conquer）也是一种解决问题的常用模式，分治法的设计思想是将无法着手解决的大问题分解成一系列规模较小的相同问题，然后逐个解决小问题，即所谓的分而治之。分治法产生的子问题与原始问题相同，只是规模减小，反复使用分治方法，可以使得子问题的规模不断减小，直到能够被直接求解为止。

应用分治法，一般出于两个目的：

- 一是通过分解问题，使无法着手解决的大问题变成容易解决的小问题；
- 二是通过减小问题的规模，降低解决问题的复杂度（或计算量）。

分治算法案例

快速排序

给 1000 个数排序，可能会因为问题的规模太大而无从下手，但是如果减小这个问题的规模，将问题一分为二，变成分别对两个拥有 500 个数的序列排序，然后再将两个排序后的序列合并成一个就得到了 1000 个数的排序结果。对 500 个数排序仍然无法下手，需要继续分解，直到最后问题的规模变成 2 个数排序的时候，只需要一次比较就可以确定顺序。这正是快速排序的实现思想，通过减小问题的规模使问题由难以解决变得容易解决。

分治算法案例

快速傅里叶变换

计算 N 个采样点的离散傅里叶变换，需要做 N^2 次复数乘法，但是将其分解成两个 $N/2$ 个采样点的离散傅里叶变换，则只需要做 $(N/2)^2+(N/2)^2=N^2/2$ 次复数乘法，做一次分解就使得计算量减少了一半，这正是快速傅里叶变换的实现思想，通过减小问题的规模减少计算量，降低问题的复杂度。

分治法的基本思想

1. **分解**

   将问题分解为若干个规模较小，相互独立且与原问题形式相同的子问题，确保各个子问题的解具有相同的子结构。

2. **求解**

   如果上一步分解得到的子问题可以解决，则解决这些子问题，否则，对每个子问题使用和上一步相同的方法再次分解，然后求解分解后的子问题，这个过程可能是一个递归的过程。

3. **合并**

   将上一步解决的各个子问题的解通过某种规则合并起来，得到原问题的解。

二分查找

```python
lis = [1, 3, 4, 6, 7, 10, 13, 14, 20, 22, 24, 30, 50, 80, 98, 100] # 有序数组
n = int(input('输入要查找的数:'))
left = 0
right = len(lis) - 1
while left <= right:
    mid = (left + right) // 2 # 整除
    if n > lis[mid]:
        left = mid + 1 # 移动左边界
    elif n < lis[mid]:
        right = mid - 1 # 移动右边界
    else:
        print("找到了，这个数在位置%s" % mid)
        break
else:
print("没有这个数据")
```

Section 5.4

递归算法

递归算法

递归算法是一种直接或者间接调用自身函数或者方法的算法。说简单了就是程序自身的调用。

递归算法实质就是将原问题不断分解为规模缩小的子问题，然后递归调用方法来表示问题的解。（用同一个方法去解决规模不同的问题）

递归作为一种算法的实现方式，与分治法天生是一对好朋友。问题的分解肯定不是一步到位的，需要反复使用分治手段，在多个层次上层层分解，这种分解的方法很自然地导致了递归方式的使用。从算法实现的角度看，分治法得到的子问题和原问题是相同的，当然可以用相同的函数来解决，区别只在于问题的规模和范围不同。通过特定的函数参数安排，使得同一个函数可以解决不同规模的相同问题，这就是递归方法的基础。

阶乘函数

阶乘函数可递归地定义为：

$$n! = \begin{cases} 1 & n = 0 \quad \text{边界条件} \\ n(n-1)! & n > 0 \quad \text{递归方程} \end{cases}$$

边界条件与递归方程是递归函数的两个要素，递归函数只有具备了这两个要素，才能在有限次计算后得出结果。

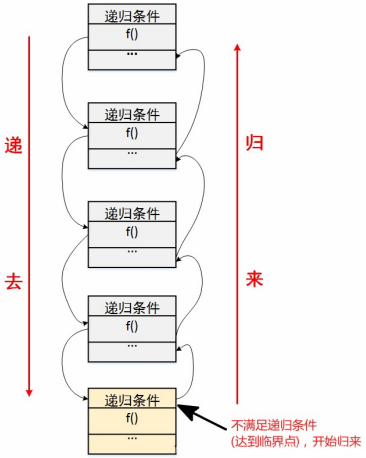
阶乘函数

```python
def factorial(n):
    if n == 0:
        return 1
return n * factorial(n-1)
```

递归的图解分析

递归算法，顾名思义就是有两个大的阶段：递和归，即有去（递去）有回（归来）。

- 递去
  将递归问题分解为若干个规模较小，与原问题形式相同的子问题，这些子问题可以用相同的解题思路来解决。
- 归来
  当你将问题不断缩小规模递去的时候，必须有一个明确的结束递去的临界点（递归出口），一旦达到这个临界点即就从该点原路返回到原点，最终问题得到解决。



斐波纳契数列

无穷数列 1，1，2，3，5，8，13，21，34，55，…，被称为 Fibonacci 数列。

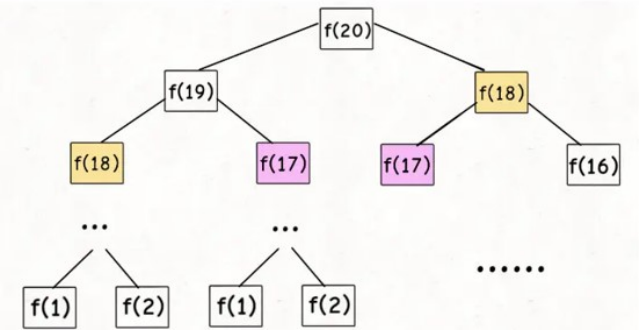
它可以递归地定义为：

$$f(n) = \begin{cases} 1 & n = 1 \quad \text{边界条件} \\ 1 & n = 2 \quad \text{边界条件} \\ f(n-1) + f(n-2) & n > 2 \quad \text{递归方程} \end{cases}$$

斐波纳契数列

```python
def fibonacci(n):
    if n <= 2:
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)
```

斐波纳契数列问题的时间复杂度



存在重叠子问题

时间复杂度 $O(2^n)$ 指数级

使用备忘录进行优化

备忘录优化

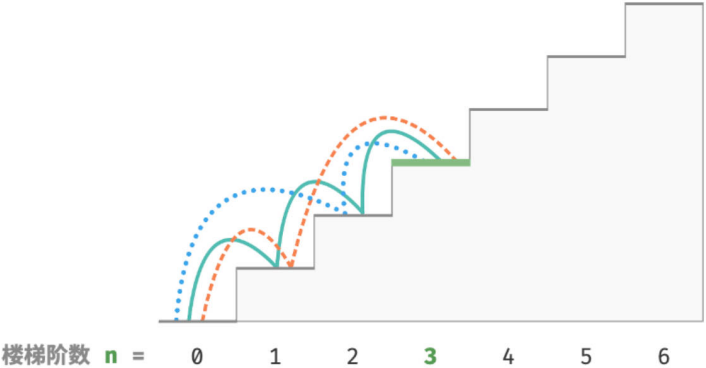
```python
memo = {0: 0, 1: 1}
def fib(n):
    if n in memo:
        return memo[n]
    memo[n] = fib(n - 1) + fib(n - 2)
    return memo[n]
```

备忘录相当于为整个递归遍历树进行了一个剪枝操作。

时间复杂度降为 $O(n)$ ，线性复杂度

例

一个台阶总共有 n 级，如果一次可以跳 1 级，也可以跳 2 级。总共有多少种跳法?



跳台阶问题

- 如果只有 1 级台阶，只有一种跳法；
- 2 级台阶有两种跳法：一种是分两次跳，每次跳 1 级；另外一种就是一次跳 2 级。
- n>2 级台阶：
  - 第一次只跳 1 级，此时跳法数目等于后面剩下的 n-1 级台阶的跳法数目，即为 f(n-1)；
  - 第一次跳 2 级，此时跳法数目等于后面剩下的 n-2 级台阶的跳法数目，即为 f(n-2);
  - 共 f(n-1) + f(n-2) 种跳法。

$$f(n)=\begin{cases}1 & n=1 \quad \text{边界条件}\\ 2 & n=2 \quad \text{边界条件}\\ f(n-1)+f(n-2) & n>2 \quad \text{递归方程}\end{cases}$$

递归和迭代的区别

“To iterate is human, to recurse, divine. —L. Peter Deutsch”

迭代的是人，递归的是神。

人的常规思维被称为递推（iterate）思维。递推是人类本能的正向思维，而“递归”则有一定的反常识。

以计算一个整数的阶乘为例，比如 5！＝1×2×3×4×5，那么做法是从小到大一个数一个数接连相乘。在生活中，这种做法不仅合情合理，而且浑然天成。

递归算法是怎么计算阶乘的呢？它是倒着来的。比如要算 5!，就把它变成 5×4！但是 4！还不知道呢！但没关系，采用同样的方法，把 4！变成 4×3！最后到 1！时，1！＝1（这就是递归的终止条件）。接下来，就是倒推回所有的结果。

“人的本质是复读机，不是挖掘机。”

Section 5.5

贪心算法

贪心算法

贪心算法，是寻找最优解问题的常用方法。这种方法模式一般将求解过程分成若干个步骤，在每个步骤都应用贪心原则，选取当前状态下最好的或最优的选择（局部最有利的选择），并以此希望最后堆叠出的结果也是最好或最优的解。贪心算法的每次决策都以当前情况为基础并根据某个最优原则进行选择，不从整体上考虑其他各种可能的情况。

贪心算法只在很少的情况下可以得到真正的最优解。大多数情况下，由于选择策略的“短视”，贪心算法会错过真正的最优解，得不到问题的真正答案。但是贪心算法简单高效，省去了为找最优解可能需要的穷举操作，可以得到与最优解比较接近的近似最优解，通常作为其他算法的辅助算法使用。

0-1 背包问题

有一个背包，最多能装下 5L 的物品，现在有 4 块蛋糕（蛋糕不能分割），要求在可以装进去的前提下，所装入的物品总价值最高。

物品	红蛋糕	黄蛋糕	绿蛋糕	白蛋糕
价值 (元)	24	10	15	25
体积 (L)	3	1	2	4

0-1 背包问题

有一个背包，最多能装下 5L 的物品，现在有 4 块蛋糕（蛋糕不能分割），要求在可以装进去的前提下，所装入的物品总价值最高。

物品	红蛋糕	黄蛋糕	绿蛋糕	白蛋糕
价值 (元)	24	10	15	25
体积 (L)	3	1	2	4

先装最小的

先装最大的

先装价值密度最大的

0-1 背包问题

有一个背包，最多能装下 5L 的物品，现在有 4 块蛋糕（蛋糕不能分割），要求在可以装进去的前提下，所装入的物品总价值最高。

物品	红蛋糕	黄蛋糕	绿蛋糕	白蛋糕
价值 (元)	24	10	15	25
体积 (L)	3	1	2	4

先装最小的

黄 + 绿 =25

先装最大的

白 + 黄 =35

先装价值密度最大的

黄 + 红 =34

0-1 背包问题

有一个背包，最多能装下 5L 的物品，现在有 4 块蛋糕（蛋糕不能分割），要求在可以装进去的前提下，所装入的物品总价值最高。

物品	红蛋糕	黄蛋糕	绿蛋糕	白蛋糕
价值 (元)	24	10	15	25
体积 (L)	3	1	2	4

先装最小的

黄 + 绿 =25

先装最大的

白 + 黄 =35

先装价值密度最大的

黄 + 红 =34

红 + 绿 =39

Section 5.6

动态规划

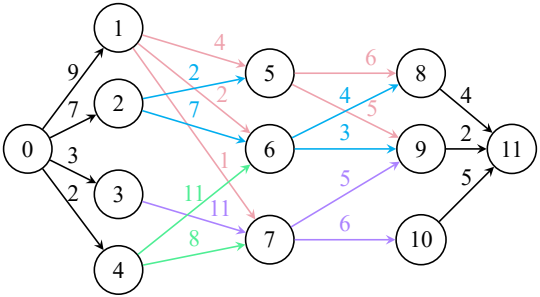
动态规划

动态规划（dynamic programming）是解决多阶段决策问题常用的最优化理论，该理论由美国数学家 Bellman 等人在 1957 年提出，用于研究多阶段决策过程的优化问题。该理论提出后，立即在数学、计算机科学、经济管理和工程技术领域得到了广泛的应用，例如最短路线、库存管理、资源分配、设备更新、排序、装载等问题，用动态规划方法往往比朴素的方法更高效。动态规划方法的原理就是把多阶段决策过程转化为一系列的单阶段决策问题，利用各个阶段之间的递推关系，逐个确定每个阶段的最优化决策，最终堆叠出多阶段决策的最优化决策结果。

生鲜运输最短路径问题

生鲜运输注重时效性，运输要求时间尽可能短。

如图所示，忽略运输其他的影响因素，运输时间最短就是距离最小。图中 0 表示生鲜农产品运输的起源地，中间可以经过 10 个地方最后才能到达目的地 11，图中的边表示两个地点之间有路，边上的数字表示两个地点的距离，求解单向通行时从源点 0 到目的地 11 的最短运输时间，限制条件不走回头路。



动态规划特点

每种方法都有自身的局限性，动态规划法也不是万能的。动态规划适合求解多阶段（状态转换）决策问题的最优解，也可用于含有线性或非线性递推关系的最优解问题，但是这些问题都必须满足最优化原理和子问题的“无后向性”。

- **最优化原理**
  不管之前决策是否是最优决策，都必须保证从现在开始的决策是在之前决策基础上的最优决策。
- **无后向性（无后效性）**
  每个阶段的决策仅受之前决策的影响，但是不影响之后各阶段的决策。

动态规划基本思想

和分治法一样，动态规划解决复杂问题的思路也是对问题进行分解，通过求解小规模的子问题再反推出原问题的结果。但是动态规划分解子问题不是简单地按照“大事化小”的方式进行的，而是沿着决策的阶段划分子问题，决策的阶段可以随时间划分，也可以随着问题的演化状态划分。分治法要求子问题是互相独立的，以便分别求解并最终合并出原始问题的解，但是动态规划法的子问题不是互相独立的，子问题之间通常有包含关系，甚至两个子问题可以包含相同的子子问题。除此之外，动态规划法的子问题还要满足“无后向性”要求。

动态规划求解步骤

1. **划分阶段**

   划分阶段没有固定的方法，根据问题的结构，可以按照时间顺序划分阶段，也可以按照问题的演化状态划分阶段。阶段划分以后，对问题的求解就变成对各个阶段分别进行最优化决策，问题的解就变成按照阶段顺序依次选择的一个决策序列。

2. **确定状态**

   对于每个阶段来说，对起始状态施加决策，使得状态发生改变，得到决策的结果状态。初始状态经过每一个阶段的决策（状态改变）之后，最终得到的状态就是问题的解。

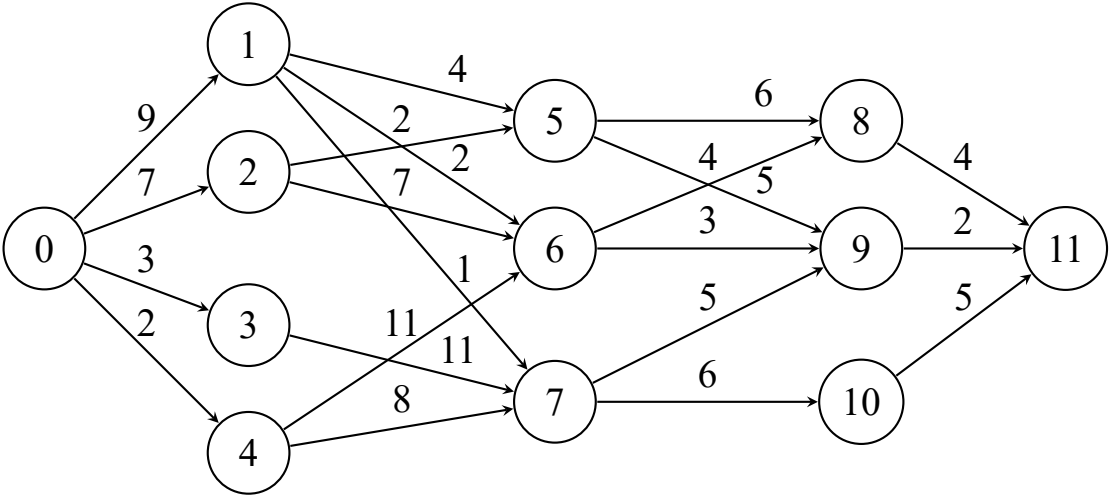
3. **确定决策并写出状态转移方程**

   决策就是能使状态发生转变的选择动作，如果选择动作有多个，则决策就是取其中能使得阶段结果最优的那一个。状态转换方程是描述状态转换关系的一系列等式，也就是从 n-1 阶段到 n 阶段演化的规律。

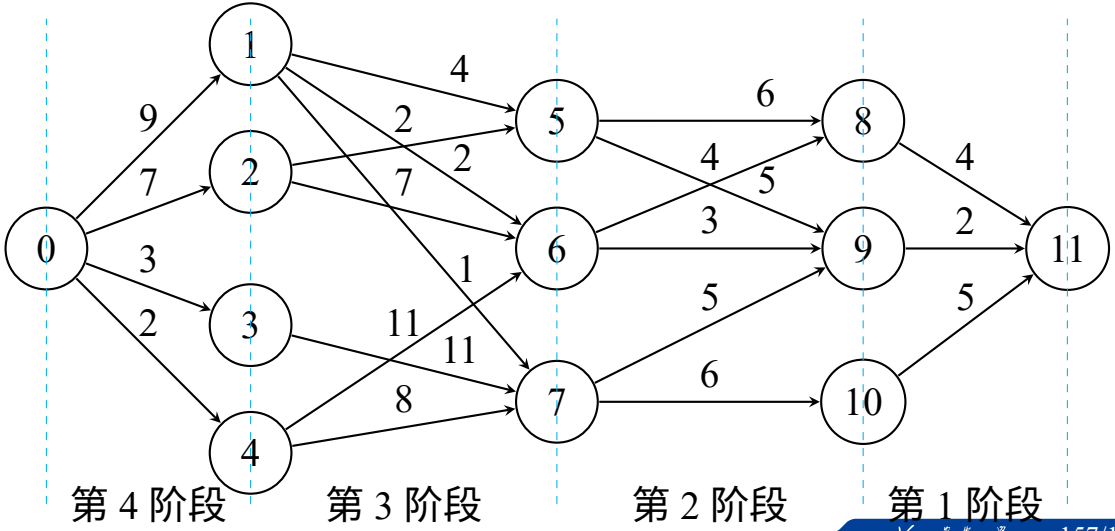
4. **确定边界条件**

   给出的状态转移方程是一个递推式，需要一个递推的终止条件或边界条件。

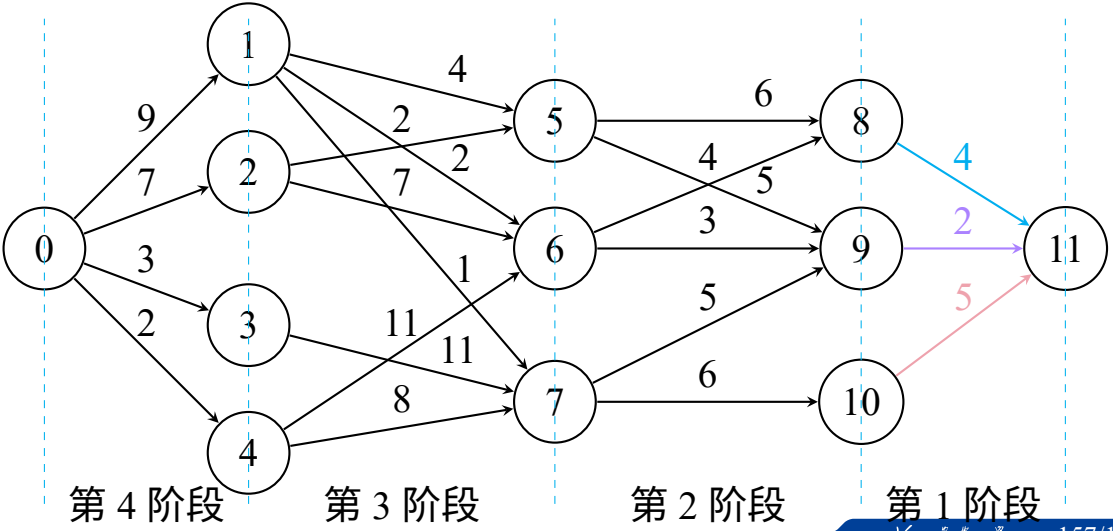
动态规划求解步骤



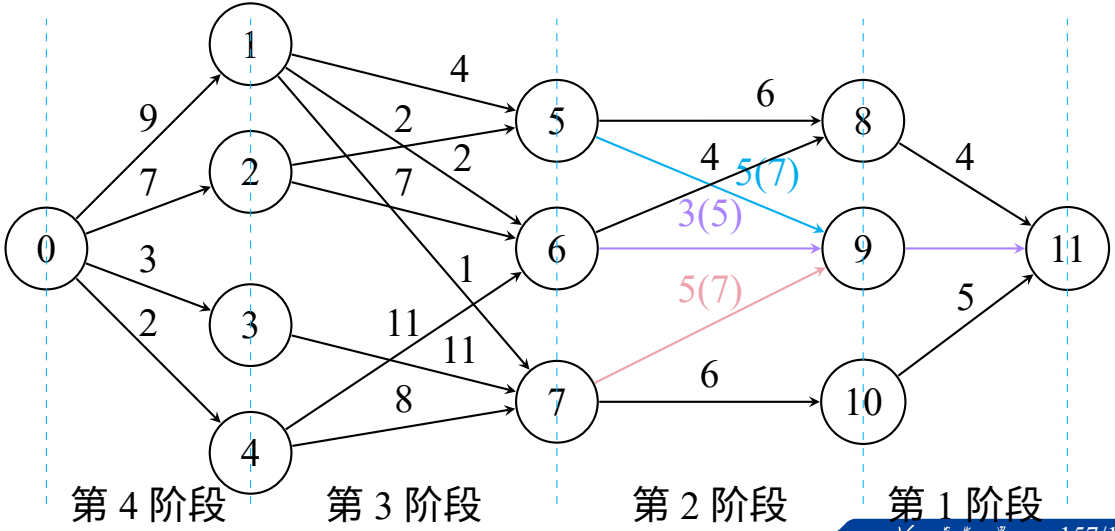
动态规划求解步骤


0
1
2
3
4
5
6
7
8
9
10
11
9
7
3
2
4
2
2
7
1
11
11
8
6
4
5
3
5
6
4
2
5
第 4 阶段
第 3 阶段
第 2 阶段
第 1 阶段

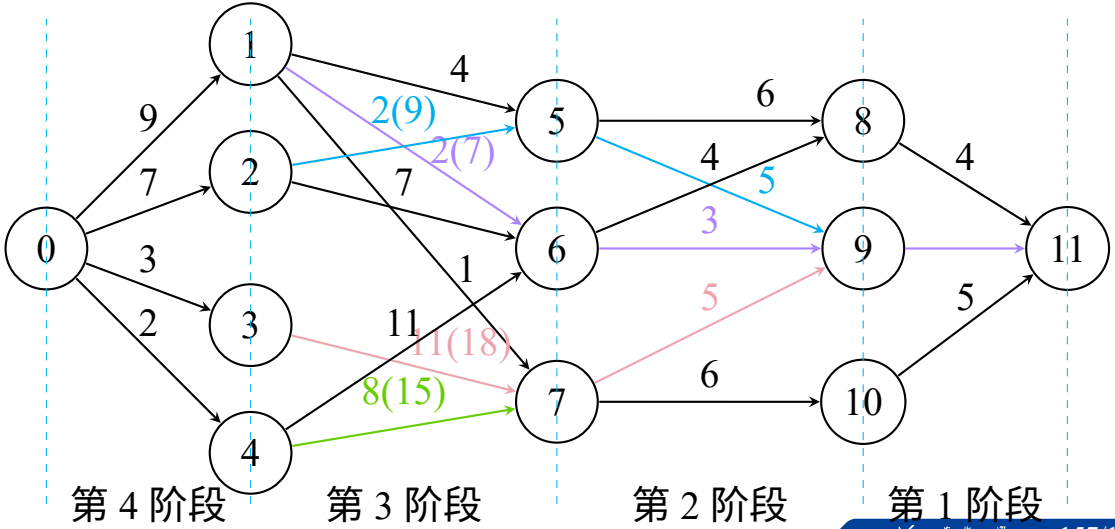
动态规划求解步骤


0
1
2
3
4
5
6
7
8
9
10
11
9
7
3
2
4
2
2
7
1
11
11
8
6
4
5
3
5
6
4
2
5
第 4 阶段
第 3 阶段
第 2 阶段
第 1 阶段

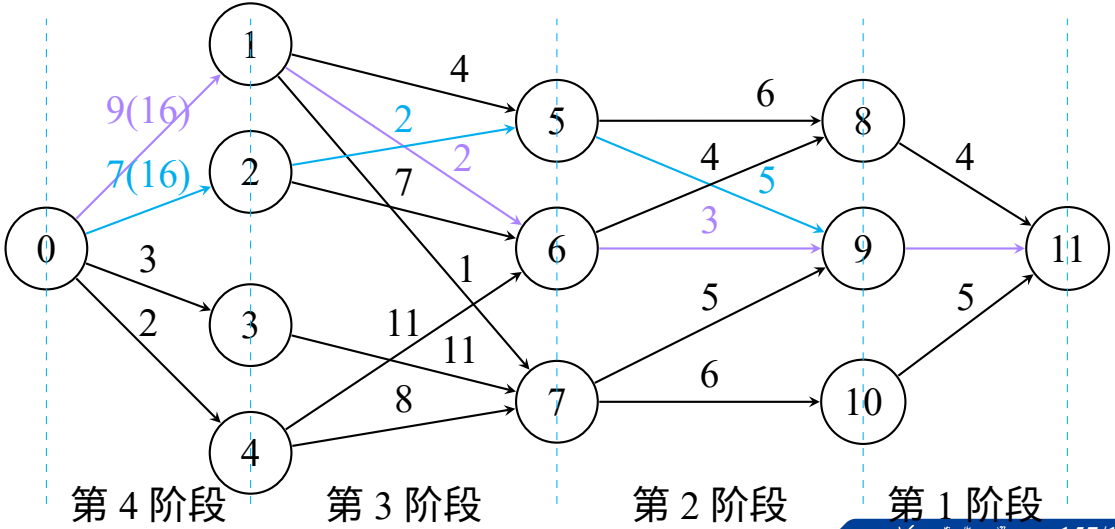
动态规划求解步骤


0
1
2
3
4
5
6
7
8
9
10
11
9
7
3
2
4
2
2
7
1
11
11
8
6
4
5(7)
3(5)
5(7)
6
4
5
第 4 阶段
第 3 阶段
第 2 阶段
第 1 阶段

动态规划求解步骤


0
1
2
3
4
5
6
7
8
9
10
11
9
7
3
2
4
2(9)
2(7)
7
1
11
11(18)
8(15)
6
4
5
3
5
6
4
5
第 4 阶段
第 3 阶段
第 2 阶段
第 1 阶段

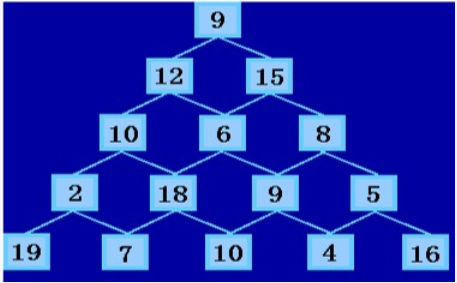
动态规划求解步骤


0
1
2
3
4
5
6
7
8
9
10
11
9(16)
7(16)
3
2
4
2
2
7
1
11
11
8
6
4
5
3
5
6
4
5
第 4 阶段
第 3 阶段
第 2 阶段
第 1 阶段

数塔问题

例

有形如图所示的一个数塔，从顶部出发，在每一结点可以选择向左走或是向右走，一直走到底层，要求找出一条路径，使路径上的数值和最大。



```python
n = 5
y = [[9], [12, 15], [10, 6, 8], [2, 18, 9, 5], [19, 7, 10, 4, 16]]
for a in range(n - 2, -1, -1):
    for b in range(a + 1):
        y[a][b] = max((y[a][b] + y[a + 1][b]), (y[a][b] + y[a + 1][b + 1])) # 父子相加取大者
        print((a, b), '=', (y[a][b]), end='-------')
    print()
num1 = y[0][0]
print('最大值为:%d' % (num1))
print('经过的数字为:', end='')
m = 0
for k in range(1, n):
    m = y[k].index(max(y[k][m], y[k][m + 1]))
    num2 = y[k][m]
    print(num1 - num2, end='->')
    num1 = num2
    if k == n - 1:
        print(num1)
```

0-1 背包问题

例

有一个背包，最多能装下 5L 的物品，现在有 4 块蛋糕（蛋糕不能分割），要求在可以装进去的前提下，所装入的物品总价值最高。

物品	红蛋糕	黄蛋糕	绿蛋糕	白蛋糕
价值 (元)	24	10	15	25
体积 (L)	3	1	2	4

0-1 背包问题

1. 如果第 i 件宝物太重，加不到背包中，那么前 i 件宝物价值等于前 i-1 件宝物价值，即 m(i, W) = m(i-1, W)；
2. 如果第 i 件宝物可以加入到背包中，那么前 i 件宝物价值等于前 i-1 件宝物价值加上第 i 件宝物价值 Wi，即 m(i, W) = m(i-1, W-Wi)+vi。

因此，m(i, W) 应该是 m(i-1, W) 和 m(i-1, W-Wi)+vi 两者最大值，即 m(i, W) = maxm(i-1, W), m(i-1, W-Wi)+vi

$$m(i,W)=\begin{cases}0 & \text{if } i=0\\ 0 & \text{if } W=0\\ m(i-1,W) & \text{if } w_i>W\\ \max\{m(i-1,W),v_i+m(i-1,W-w_i)\} & \text{otherwise}\end{cases}$$

动态规划表格

w →

i ↓

m	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	24	24	24
2	0	10	10	24	24	34
3	0	10	15	25	34	39
4	0	10	15	25	34	39

装载宝物

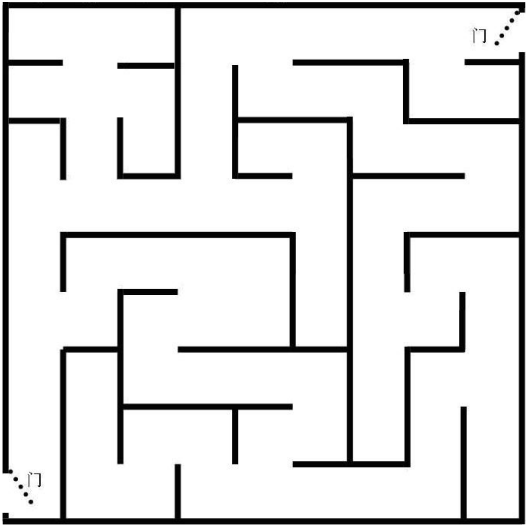
```python
goods = [None, {'w': 3,'v': 24}, {'w': 1,'v': 10}, {'w': 2,'v': 15}, {'w': 4,'v': 25}]
max_w = 5 # 设置背包最大承重
# 初始化二维表格m[(i, w)]，将表格中所有价值均初始化为0
m = {(i, w): 0 for i in range(len(goods)) for w in range(max_w + 1)}
for i in range(1, len(goods)):
    for w in range(1, max_w + 1):
        if goods[i]['w'] > w:
            # 装不下第i个宝物，即不装第i个宝物
            m[(i, w)] = m[(i - 1, w)]
        else:
            # 装得下第i个宝物时，在不装第i个宝物与装第i个宝物这两种情况下，取最大价值
            m[(i, w)] = max(m[(i - 1, w)], m[(i - 1, w - goods[i]['w'])] + goods[i]['v'])
print(m[(len(goods) - 1, max_w)]) # 输出结果
```

Section 5.7

回溯法

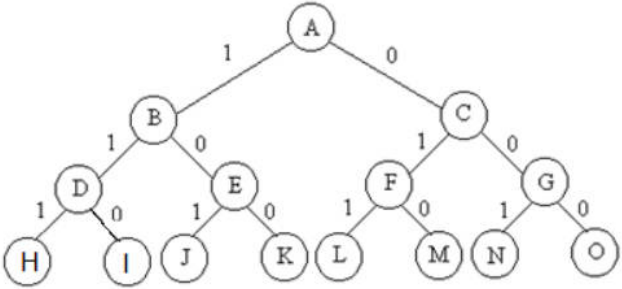
回溯法

「回溯算法」是一种通过穷举来解决问题的方法，它的核心思想是从一个初始状态出发，暴力搜索所有可能的解决方案，当遇到正确的解则将其记录，直到找到解或者尝试了所有可能的选择都无法找到解为止。当算法在搜索过程中遇到某个状态无法继续前进或无法得到满足条件的解时，它会撤销上一步的选择，退回到之前的状态，并尝试其他可能的选择。

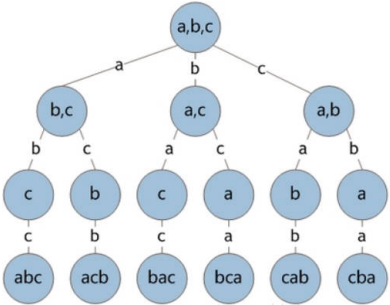


问题解空间

应用回溯法求解问题时，首先应明确定义问题的解空间，该解空间应至少包含问题的一个最优解。在定义了问题的解空间后，还需要将解空间有效地组织起来，使得回溯法能方便地搜索整个解空间，通常将解空间组织成子集树和排列树的形式。



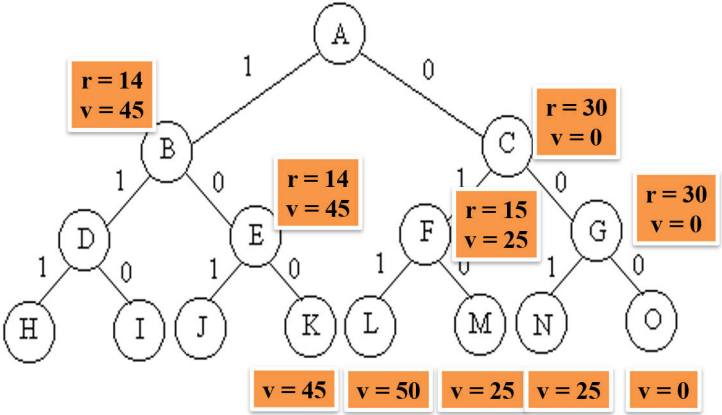
(a) 子集树



(b) 排列树

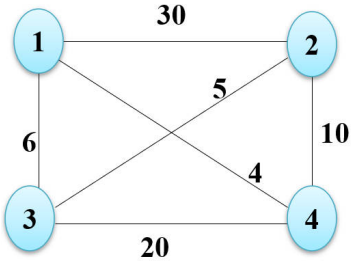
0-1 背包问题

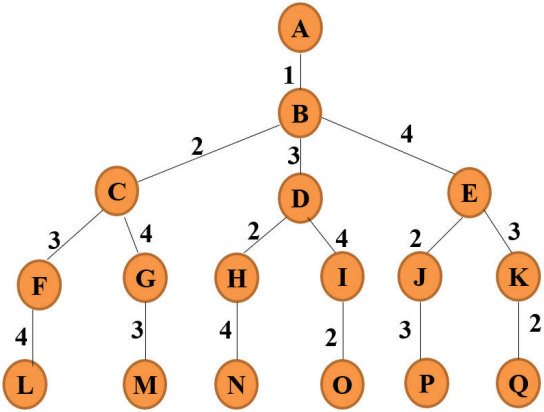
给定一背包的容量 C，和 N 个物品的重量 w_i 价值 v_i，求在背包容量允许的条件下能装入的最大价值。设 $N = 3, W = (16, 15, 15), P = (45, 25, 25), C = 30$



旅行商问题

某售货员要到若干城市去推销商品，已知各城市之间的路程，他要选定一条从驻地出发，经过每个城市一遍，最后回到驻地的路线，使总的路程最短。





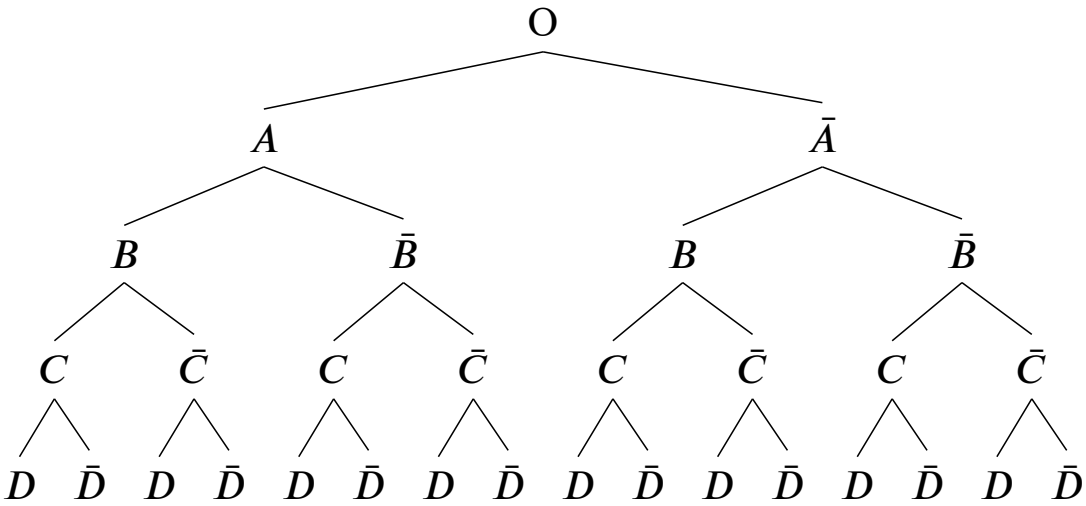
Section 5.8

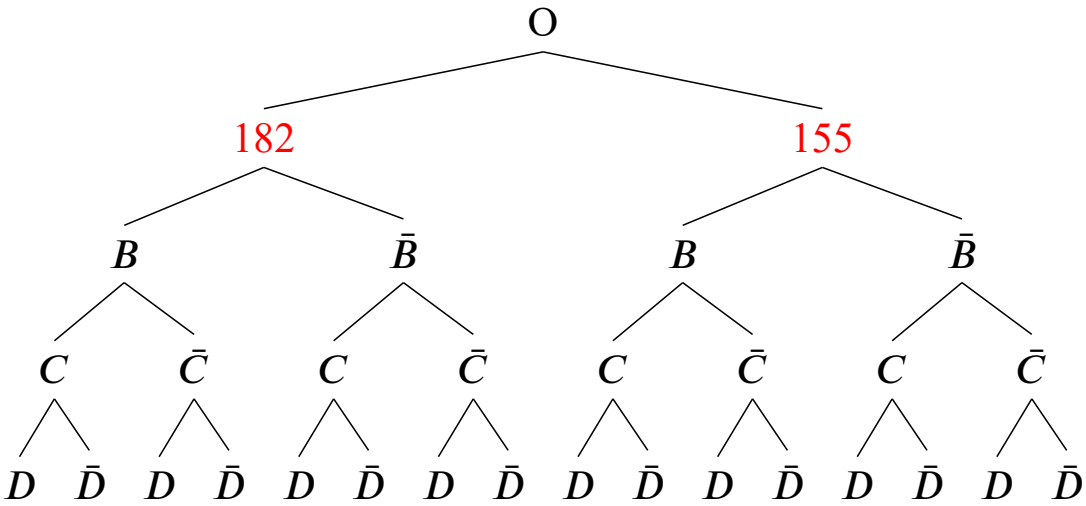
分支限界法

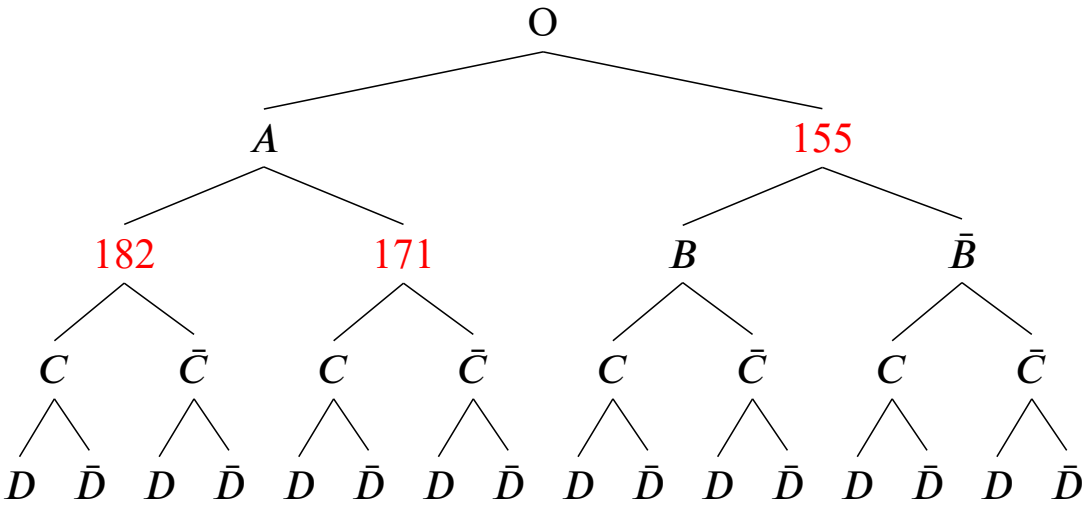
分支限界法

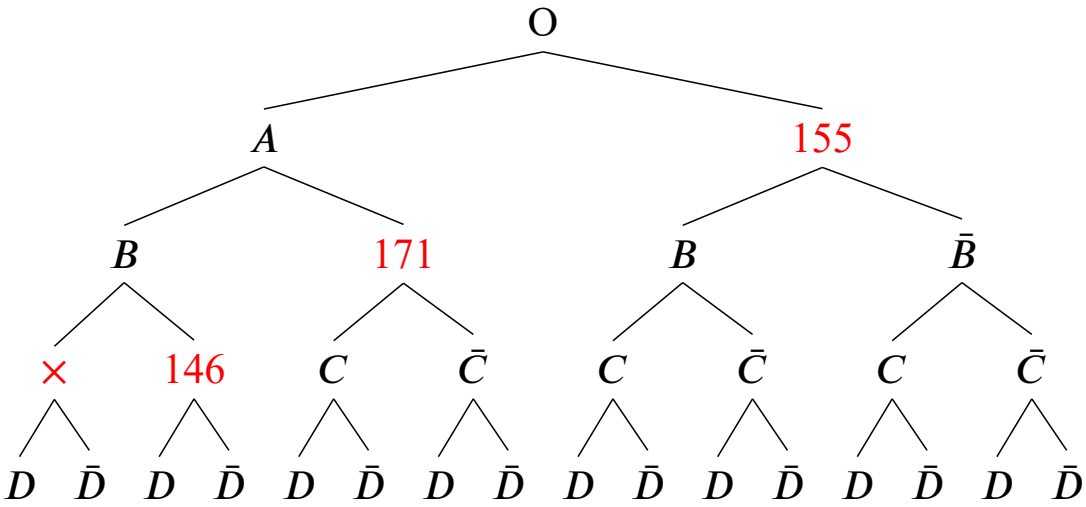
例

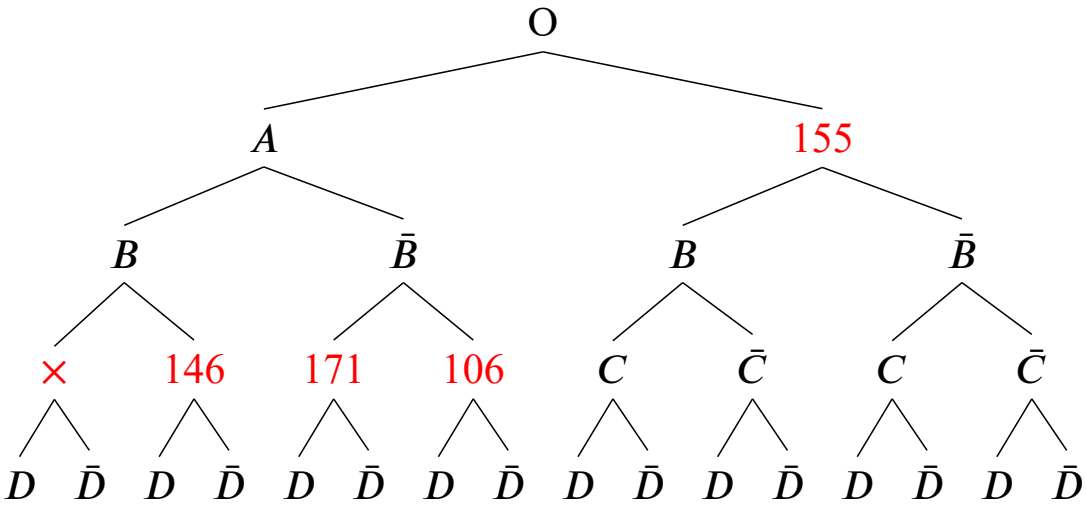
假定有 N=4 件商品，分别用 A 、B 、C 、D 表示。每件商品的重量分别为 3kg 、2kg 、5kg 和 4kg ，对应的价值分别为 66 元、40 元、95 元和 40 元。现有一个背包，可以容纳的总重量为 9kg ，问：如何挑选商品，使得背包里商品的总价值最大?

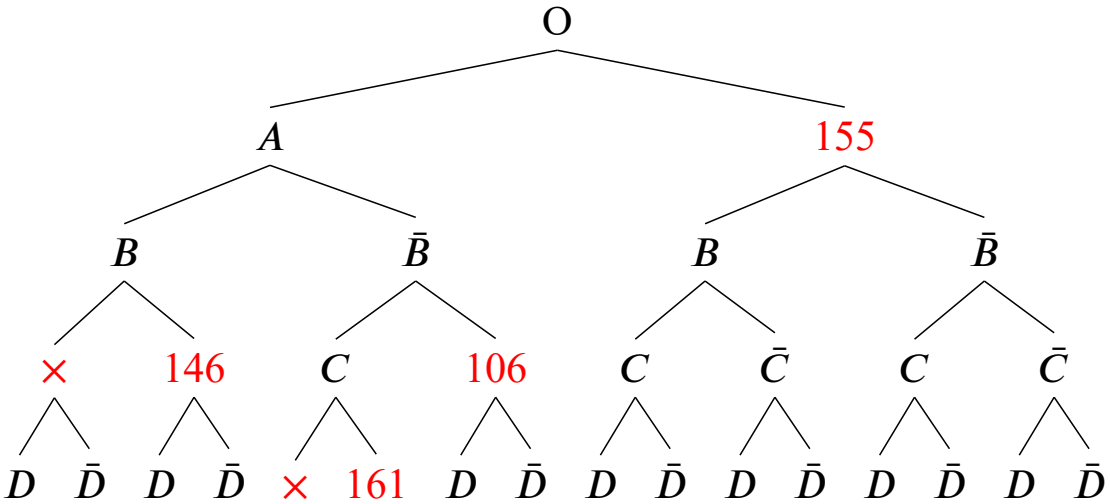

O
A
$\bar{A}$
B
$\bar{B}$
B
$\bar{B}$
C
$\bar{C}$
C
$\bar{C}$
C
$\bar{C}$
C
$\bar{C}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$


O
182
155
B
$\bar{B}$
B
$\bar{B}$
C
$\bar{C}$
C
$\bar{C}$
C
$\bar{C}$
C
$\bar{C}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$


O
A
155
182
171
B
$\bar{B}$
C
$\bar{C}$
C
$\bar{C}$
C
$\bar{C}$
C
$\bar{C}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$


O
A
155
B
171
B
$\bar{B}$
×
146
C
$\bar{C}$
C
$\bar{C}$
C
$\bar{C}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$


O
A
155
B
$\bar{B}$
B
$\bar{B}$
×
146
171
106
C
$\bar{C}$
C
$\bar{C}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$


O
A
155
B
$\bar{B}$
B
$\bar{B}$
×
146
C
106
C
$\bar{C}$
C
$\bar{C}$
D
$\bar{D}$
D
$\bar{D}$
×
161
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$
D
$\bar{D}$